Parallel Decoding of Language Models: A Survey

Yu-Yang Qian, Jia-Chen Liu, Hao-Cong Wu, Lanxiang Hu, Yibo Zhu, Hao Zhang, and Peng Zhao

***Abstract*—Large language models (LLMs) have achieved remarkable success; however, their autoregressive (AR) decoding mechanism generates tokens one at a time, making the inherent sequential dependency a dominant latency bottleneck in practical deployment. A rapidly growing body of work, termed as *parallel decoding*, seeks to break this sequential barrier by introducing parallelism while preserving output quality. Despite growing research interest, the diverse and fast-evolving landscape of parallel decoding methods still lacks a unified taxonomy. In this survey, we present a comprehensive and systematic review of parallel decoding, organized by the *granularity at which parallelism is introduced*, ranging from tokens to sequences and tasks. Specifically, we organize existing work into seven categories, spanning from fine-grained token-level parallelism to coarse-grained task-level orchestration: (i) *token-level* methods that generate multiple tokens in parallel; (ii) *sequence-level* methods that parallelize across reasoning steps and semantic units; (iii) *task-level* methods that enable concurrent task execution and tool invocation; (iv) *architecture-level* methods that replace autoregressive generation with inherently parallel paradigms such as discrete diffusion language models; (v) *system-level* optimizations that integrate parallel decoding into production serving infrastructures; (vi) *hybrid* methods that combine techniques across multiple levels; and (vii) *benchmarks and evaluation* resources tailored to parallel decoding. We further report benchmark results that validate the effectiveness of representative methods. By providing a structured and unified overview of the parallel decoding landscape, this survey aims both to serve as an accessible guide for researchers and to identify open challenges and promising future directions.**

***Index Terms*—Large Language Models, Parallel Decoding, Speculative Decoding, Efficient Inference**

I. Introduction

LARGE language models (LLMs) have achieved remarkable success across a wide range of tasks, from natural language understanding and generation [1, 2, 3] to code synthesis [4, 5] and multimodal reasoning [6]. However, this progress comes at the cost of growing computational demands. State-of-the-art models now comprise hundreds of billions of parameters [7, 8], and emerging techniques such as chain-of-thought prompting [9], multi-step reasoning [10], and long-horizon agentic workflows [11] substantially increase the number of tokens generated per request. The resulting inference burden has become a major obstacle to efficient deployment.

A variety of approaches have been proposed to address these efficiency challenges, including model compression techniques such as quantization [12] and pruning [13], efficient attention mechanisms [14] and sparse attention [15], distributed serving systems [16], and system-level serving optimizations [17]. Among these directions, this survey focuses on *parallel decoding*: a general and widely applicable family of techniques that aim to break the sequential bottleneck inherent in autoregressive token generation.

Specifically, the root cause of inference inefficiency lies in the autoregressive (AR) decoding mechanism. Standard AR decoding generates tokens one at a time: each new token depends on all preceding tokens and cannot be produced until the previous one is fully generated. This sequential dependency leaves the parallel processing capabilities of modern hardware accelerators underutilized, making inference a memory-bandwidth-bound process rather than a compute-bound one. Recognizing this limitation, a growing body of work referred to as *parallel decoding*, has attempted to go *beyond* the one-token-at-a-time paradigm by introducing parallelism into the decoding process. For example, speculative decoding [18, 19] employs a lightweight draft model to propose multiple candidate tokens that are verified in parallel by the target model; multi-token prediction [20, 7] augments the model with additional prediction heads to produce multiple tokens; and diffusion language models [21, 22] replace sequential generation with an inherently parallel mask-and-denoise process. In this survey, we present a comprehensive and systematic review of parallel decoding methods. We introduce a unified taxonomy organized by the *granularity* at which parallelism is introduced, spanning from individual tokens to entire tasks. Specifically, we categorize existing work into seven classes:

(1) *Token-level parallel decoding* (Section III), which generates and verifies multiple candidate tokens within a single decoding step through draft-then-verify mechanisms;
(2) *Sequence-level parallel decoding* (Section IV), which parallelizes generation across multiple reasoning paths or structurally independent segments of a single output;
(3) *Task-level parallel decoding* (Section V), which enables concurrent execution of higher-level units such as agentic orchestration and tool invocation;
(4) *Architecture-level parallel decoding* (Section VI), which replaces autoregressive generation with inherently parallel architectures and their associated inference algorithms;
(5) *System and infrastructure* (Section VII), which optimizes serving, scheduling, and hardware co-design for deploying parallel decoding in production;
(6) *Hybrid methods* (Section VIII), which combine techniques across the different categories;
(7) *Benchmarks and evaluation* (Section IX), which encompass related surveys and benchmarks.

Finally, we present experimental comparisons of representative methods in Section X. Figure 1 illustrates the proposed taxonomy. By providing a structured and up-to-date overview

Y.-Y. Qian, J.-C. Liu, H.-C. Wu, and P. Zhao are with State Key Laboratory for Novel Software Technology and School of Artificial Intelligence, Nanjing University, Nanjing, China. Y. Zhu is with StepFUN, China. L. Hu and H. Zhang are with University of California San Diego, San Diego, USA.

Preprint.

of the parallel decoding landscape, this survey aims to serve as both an accessible guide for newcomers and a useful reference for experienced researchers, while highlighting open challenges and promising directions for future work.

Organization. Section II provides an overview of parallel decoding and introduces our taxonomy. Sections III to IX review representative methods at each granularity level. Section X presents experimental comparisons of representative methods. Section XI concludes the paper.

II. Overview

In this section, we provide an overview of parallel decoding. We first describe the inference pipeline of autoregressive LLMs and analyze why the sequential decoding process becomes a bottleneck on modern hardware. We then introduce the key ideas behind parallel decoding and outline the taxonomy adopted throughout this survey.

A. Sequential Dependencies in AR Models

Modern LLMs are built on the transformer architecture and generate text through *autoregressive* (AR) decoding. Given a vocabulary $\mathcal{V}$ and model parameters θ, the model defines a conditional distribution over the next token given all preceding tokens. The inference pipeline of LLMs consists of two phases:

Prefill phase. Given an input prompt $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathcal{V}^n$, the model processes all n tokens in a *single* forward pass. At each transformer layer $l \in \{1, \ldots, L\}$, the model computes the key and value representations for every input position and stores them in the *key-value (KV) cache*:

$$\mathbf{K}^{(l)} = [\mathbf{k}_1^{(l)}, \ldots, \mathbf{k}_n^{(l)}], \quad \mathbf{V}^{(l)} = [\mathbf{v}_1^{(l)}, \ldots, \mathbf{v}_n^{(l)}].$$

Since all prompt tokens are available in advance, the computation can be fully parallelized across the n positions, achieving high utilization of modern accelerators.

Decoding phase. Starting from the prompt, the model generates output tokens $\mathbf{y} = (y_1, y_2, \ldots, y_m)$ one at a time. At each step t, the model takes the most recently generated token y_{t-1} as input, computes its key-value pair and appends it to the KV-cache, attends to all cached representations from preceding positions, and produces a probability distribution from which the next token y_t is sampled:

$$y_t \sim p_\theta(y_t \mid x_1, \ldots, x_n, y_1, \ldots, y_{t-1}), \quad t = 1, \ldots, m. \quad (1)$$

This process repeats until a stopping condition is met (e.g., an end-of-sequence token, i.e., EOS, is generated or the maximum length is reached). Critically, the generation of y_{t+1} cannot begin until y_t is fully produced, because y_{t+1} depends on y_t through both the conditional distribution in Eq. (1) and the updated KV-cache. This strict sequential dependency is the fundamental bottleneck of AR decoding.

Why is sequential decoding slow on modern GPUs? Sequential decoding makes poor use of modern GPUs. Each decoding step produces only a single token, yet it must load essentially all model weights from memory, where each loaded weight is used for only a few floating-point operations. The resulting arithmetic intensity, i.e., the ratio of computation performed to bytes of memory accessed, is far below what modern accelerators are designed for, leaving most compute units idle. Decoding is therefore *memory-bandwidth bound* rather than compute-bound, particularly in the small-batch regime typical of latency-sensitive serving; only very large batches or long contexts shift the bottleneck toward compute. We define the *parallelism* of a decoding method as:

$$\text{Parallelism} \triangleq \frac{S}{s},$$

where S is the number of decoding (or execution) steps under standard autoregressive decoding, and s is the number of sequential model calls that the parallel decoding method needs to produce an output of the same length. Standard AR decoding has $s = S$ and hence a parallelism of 1; parallel decoding methods reduce s by processing multiple tokens, sequences, or tasks concurrently. For example, speculative decoding that accepts on average k draft tokens per verification step attains a parallelism of about k (in expectation, since the number of accepted tokens per round is random). This definition applies most directly to output-preserving token-level methods; for sequence- and task-level methods, S and s are interpreted as the critical-path lengths of the serial and the parallelized execution of the same workload. Higher parallelism improves hardware utilization and, when the overheads of drafting, verification, and synchronization are moderate, reduces end-to-end latency, which is the central goal of parallel decoding.

B. Beyond Autoregression: Parallel Decoding Methods

To address the sequential bottleneck analyzed above, a growing body of research has explored methods that generate or verify multiple tokens within a single decoding step, thereby amortizing the per-token memory access cost and improving GPU utilization. Among these, one representative approach is *speculative decoding* [18, 19], which follows a *draft-then-verify* paradigm: a lightweight draft model first proposes a candidate sequence of k tokens, and the larger target model then verifies all k candidates in a single forward pass. In the memory-bandwidth-bound regime (small batches and moderate draft lengths), this verification costs roughly the same as processing a single token when KV caching is used. Draft tokens accepted by the verification rule are retained; upon the first rejection, a replacement token is sampled from a corrected distribution and generation continues from that position, ensuring that the output distribution is exactly preserved.

Another prominent direction to address the sequential bottleneck is *multi-token prediction* (MTP) [20, 7], which augments the model with additional prediction heads to produce multiple future tokens. More recently, *diffusion language models* [21, 22] have emerged as a fundamentally different paradigm that replaces sequential generation entirely with a parallel mask-and-denoise process.

Despite their diverse mechanisms, these methods share a common objective: replacing serial one-token-at-a-time generation with parallel operations to reduce the number of decoding steps and increase computational utilization. In this survey, we observe that the key distinction among them lies in the *granularity* at which parallelism is introduced. Some methods

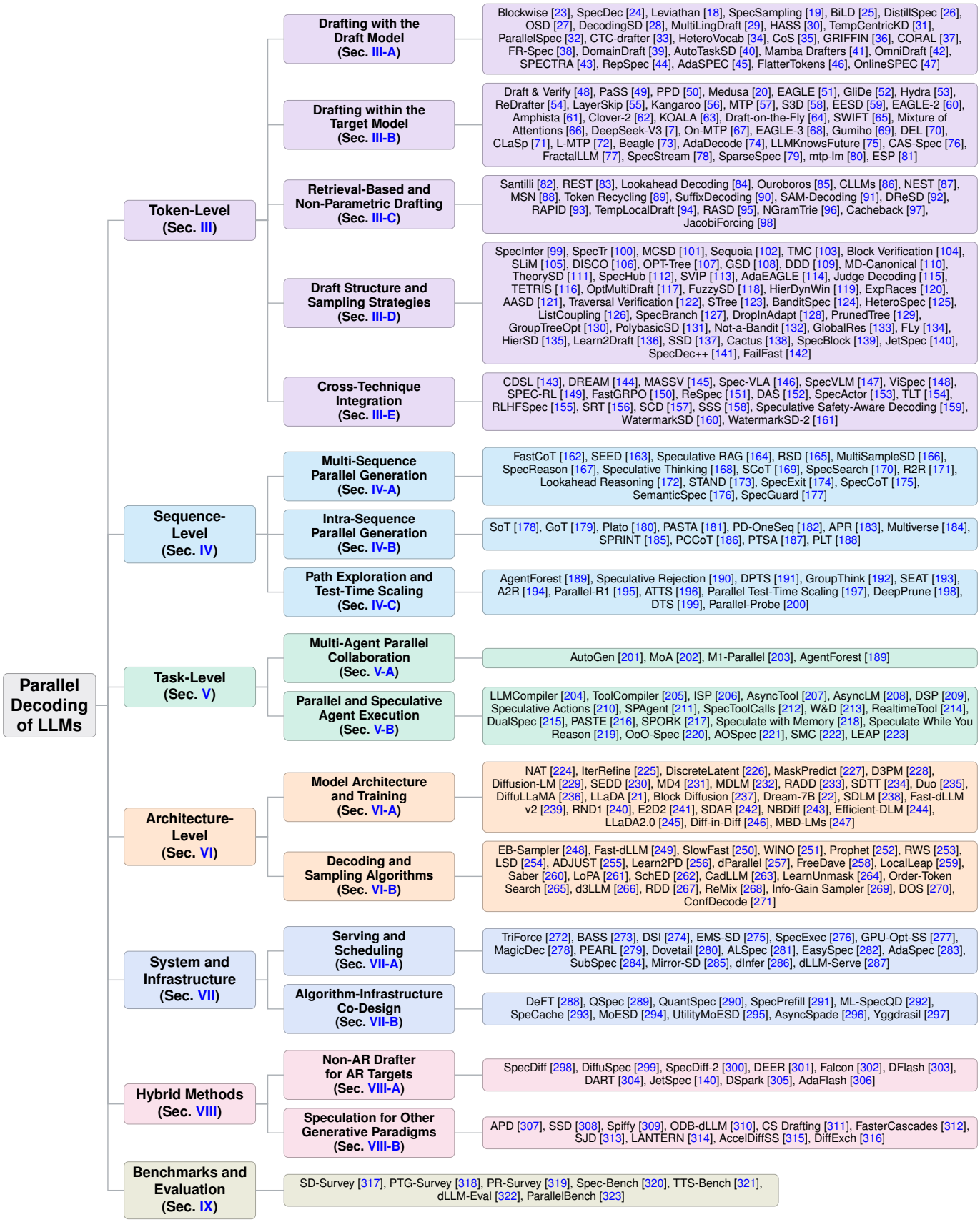


Fig. 1: Taxonomy of parallel decoding methods. We organize existing work by the *granularity* at which parallelism is introduced, i.e., from individual tokens, through sequences and tasks, to model architectures, and further include system-level infrastructure, hybrid methods, and benchmarks. Each granularity level is subdivided into finer categories based on how parallelism is realized within that level.

operate at the level of individual *tokens* (e.g., speculative decoding generates and verifies multiple token candidates in parallel), while others introduce parallelism at the level of reasoning *sequences* (e.g., exploring multiple reasoning paths concurrently) or entire *tasks*. Beyond algorithmic innovations, some approaches redesign the *model architecture* itself to support inherently parallel generation (e.g., diffusion language models), while *system-level* techniques optimize the serving and infrastructure stack to deploy these methods at scale.

In the following sections, we present a systematic taxonomy of parallel decoding methods organized by this *granularity of parallelism*. As summarized in Figure 1, we categorize existing work into seven classes: token-level, sequence-level, task-level, architecture-level, system and infrastructure, hybrid methods, and benchmarks.

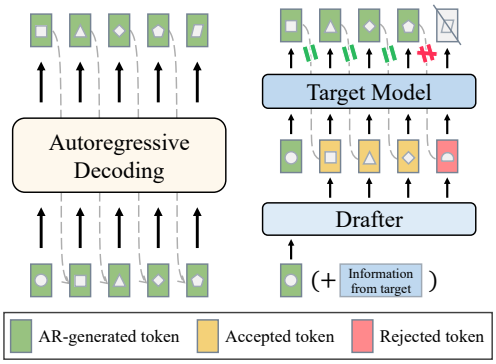


Fig. 2: Illustration of *token-level parallel decoding*. Standard autoregressive decoding generates one token per step (left). Token-level methods (right) first generate multiple candidate tokens, then verify them using target model to preserve the output tokens' distribution.

III. Token-Level Parallel Decoding

In this section, we review *token-level parallel decoding* methods, which generate and verify multiple draft token candidates within a single decoding step, as illustrated in Figure 2. These methods represent one of the most widely studied and commonly adopted approaches to parallel decoding. We organize them into five subcategories based on how draft token candidates are produced and processed: drafting with an independent draft model (Section III-A), drafting within the target model (Section III-B), retrieval-based and non-parametric drafting (Section III-C), draft structure and sampling strategies (Section III-D), and cross-technique integration (Section III-E).

A. Drafting with the Draft Model

The most direct approach to token-level parallel decoding employs a separate lightweight model, commonly called the *draft model*, to propose multiple candidate tokens that the larger *target model* then verifies in parallel. The idea of predicting multiple future positions and retaining only the longest validated prefix dates back to Stern et al. [23]. Building on this, a cluster of works in 2023 established the modern *draft-then-verify* paradigm. Xia et al. [24] paired an optimized drafter with an efficient verification strategy, achieving around $5\times$ speedup on seq2seq tasks. Leviathan et al. [18] and Chen et al. [19] independently proposed *lossless* speculative decoding, whose rejection-sampling verification provably preserves the target model's output distribution without retraining or architecture changes. BiLD [25] further introduced fallback and rollback policies to coordinate a small and a large model for plug-and-play acceleration.

A central challenge in this paradigm is aligning the draft model's distribution with the target model to achieve a high acceptance rate. Knowledge distillation is widely used for this purpose. DistillSpec [26] improves alignment through on-policy data generation and task-specific divergence functions. Ouyang et al. [31] showed that consistent-temperature distillation effectively addresses performance degradation under high-temperature generation. AdaSPEC [45] introduces selective token filtering to focus distillation on tokens the draft model can learn effectively. Hong et al. [39] found that offline distillation outperformed online distillation. Fan et al. [46] observed that tokens inducing flatter target distributions contribute more to the acceptance rate, and proposed a flatness-based dataset distillation approach accordingly.

Several studies have examined draft model design from broader perspectives. Yan et al. [28] found that draft model latency matters more than language modeling capability, motivating hardware-efficient draft architectures. HASS [30] addresses the discrepancy between training and decoding contexts through harmonized representation learning. A separate direction focuses on *continuous adaptation* of the draft model during deployment. OSD [27] pioneers this by continuously updating the draft model on observed user queries to mitigate distribution shift. OnlineSPEC [47] further provides a principled online learning framework with theoretical guarantees connecting dynamic regret to the acceleration rate.

Beyond training, a parallel line of research focuses on draft model *architecture*. ParallelSpec [32] trains a non-autoregressive drafter to predict multiple tokens simultaneously, removing the sequential dependency in drafting. CTC-drafter [33] employs a CTC-based architecture that models correlations among draft tokens for higher-quality candidates. Mamba-based drafters [41] leverage the linear complexity of state space models to reduce both latency and memory consumption. On the training-inference alignment side, GRIFFIN [36] introduces token-alignable training with loss masking and input token correction, CORAL [37] enhances multi-step consistency through cross-step representation alignment, and RepSpec [44] applies structural re-parameterization to improve training without additional inference cost. CoS [35] extends speculative decoding to a collaborative multi-model setting with provable speedup guarantees, and SPECTRA [43] combines internal and external speculation in a training-free manner.

A practical limitation of early methods is the requirement that draft and target models share the same vocabulary and domain. Yi et al. [29] showed that language-specific draft models substantially improve inference speed in multilingual settings. Timor et al. [34] removed the shared-vocabulary constraint entirely, enabling any off-the-shelf model to serve

as a drafter. FR-Spec [38] compresses the vocabulary space by constraining draft search to a frequency-prioritized token subset. OmniDraft [42] unifies cross-vocabulary adaptation and online learning to enable one drafter to serve diverse target models from different families. At the task level, Ge et al. [40] proposed automatic task detection to route prompts to heterogeneous draft models aligned with task-specific data.

B. Drafting within the Target Model

An alternative to maintaining a separate draft model is to produce drafts *within* the target model itself, either by augmenting it with lightweight prediction modules or by reusing subsets of its own layers. The most common strategy attaches extra prediction heads or input tokens. PaSS [49] introduces an additional input token to mark positions for parallel generation, and PPD [50] appends trained prompt tokens to approximate future outputs. Medusa [20] augments the target LLM with multiple extra decoding heads combined with tree-based attention for verification. Hydra [53] extends Medusa with sequentially-dependent draft heads that condition on preceding draft tokens. Gumiho [69] proposes a hybrid architecture combining serial Transformer heads for early positions with parallel MLP heads for later ones. Other works explore bi-directional attention [61], adversarial training [63], and multi-stream attention [78] for prediction head design.

Rather than predicting tokens directly, another line of work performs autoregression at the *feature level*, leveraging the target model's intermediate representations. EAGLE [51] predicts second-to-top-layer features with token-conditioned drafting. GliDe with a CaPE [52] reuses cached keys and values from the target LLM with confidence-based candidate selection. EAGLE-2 [60] introduces context-aware dynamic draft trees based on confidence scores. ReDrafter [54] employs an RNN-based draft model with dynamic tree attention. Mixture of Attentions [66] addresses limitations such as the lack of on-policyness and partial observability, and extends to client-server scenarios. Lightweight RNN-based [62] and cross-attention-based [73] designs have also been explored. EAGLE-3 [68] advances to direct token prediction with multi-layer feature fusion for better data scalability.

A complementary direction augments the training objective to support *multi-token prediction* (MTP), enabling the model to produce multiple future tokens. The MTP framework [57] trains independent output heads on a shared trunk for parallel token prediction. DeepSeek-V3 [7] adopts sequential MTP modules with complete causal chains; while designed for training, they can also serve for speculative decoding at inference time. Further study [67] reveals that pretrained models inherently possess MTP capabilities, though their representations are specialized for single-token prediction. L-MTP [72] captures longer-range dependencies by predicting non-sequential tokens that skip intermediate positions. Recent work [75] shows that autoregressive models possess latent future-token knowledge and proposes gated LoRA to unlock it. Self-distillation [80] and embedding-space probing [81] have also been explored for eliciting MTP capabilities.

Self-speculative decoding takes a different approach by reusing subsets of the target model's own layers for both drafting and verification. Draft & Verify [48] generates drafts by skipping intermediate layers, requiring no additional training or memory. LayerSkip [55] uses layer dropout and early exit loss to enable accurate early-layer predictions. EESD [59] combines early-exiting with self-distillation and Thompson sampling for adaptive draft length. Kangaroo [56] trains a lightweight adapter on a shallow sub-network with a double early exiting mechanism. Other works explore compressed layers [77], layer skipping with multi-token decoding [58], and sparse attention with system-level co-design [79].

While the above methods often require training or a fixed exit configuration, recent work explores *plug-and-play* approaches that adaptively determine layer skipping or early exit without training. Draft-on-the-Fly [64] adapts draft models to input context via cosine similarity. SWIFT [65] exploits task-specific layer sparsity for on-the-fly layer skipping. CLaSp [71] optimizes the layer-skipping strategy via dynamic programming after each verification. DEL [70] dynamically selects exit layers and speculation lengths based on tracked acceptance rates. AdaDecode [74] generates tokens at intermediate layers when confidence is high, deferring remaining computations to run in parallel with subsequent tokens. Cascade-based routing among multiple self-speculative strategies has also been explored [76].

C. Retrieval-Based and Non-Parametric Drafting

Unlike the methods discussed above, which rely on separate learned draft models or dedicated prediction modules, a separate family of approaches obtains draft tokens either from fixed-point iteration on the target model itself or from retrieval over existing text, we regard both as *non-parametric* drafting in the sense that no separate draft model is introduced. A prominent direction applies *Jacobi iteration*, a fixed-point method that initializes a sequence of tokens and iteratively refines them in parallel until convergence. Santilli et al. [82] propose to reframe autoregressive decoding as Jacobi and Gauss–Seidel fixed-point iteration for parallel inference. Lookahead Decoding [84] extends this idea by maintaining n-gram pools collected from the Jacobi decoding trajectory, which serve as candidate tokens for parallel verification, eliminating the need for any auxiliary model or data store. CLLMs [86] further improve convergence speed by training the target model to consistently predict the fixed point from any intermediate state on the Jacobi trajectory. MSN [88] introduces a noisy training framework that enhances the model's parallel decoding capability by learning a denoising task, and proposes a retrieval-augmented Jacobi decoding strategy for further acceleration. More recently, Jacobi Forcing [98] applies progressive distillation where models are trained on their own parallel decoding trajectories, smoothly shifting autoregressive models into efficient parallel decoders while preserving causal inference.

Another line of work retrieves draft tokens from external data sources rather than generating them through model computation. REST [83] introduces retrieval-based drafting, retrieving candidate tokens from a non-parametric datastore based on the current context and offering a plug-and-play

solution that accelerates any language model without additional training. NEST [87] extends this approach to a semi-parametric setting, performing token-level retrieval at each step to compute a mixture distribution and identify promising span continuations from a corpus, enabling both acceleration and source attribution. DReSD [92] replaces the sparse string matching used in REST with dense retrieval based on contextualized token embeddings, achieving substantially higher acceptance rates through more semantically relevant token retrieval. RASD [95] combines retrieval-based and model-based drafting by merging their respective draft trees through tree pruning and longest prefix matching, achieving robust acceleration across both in-domain and out-of-domain scenarios. Retrieval-augmented drafting has also been applied to long-context inference, where a draft model operating on shortened retrieval contexts can serve as an efficient drafter for the full-context target model [93].

A closely related family of methods reuses tokens generated during the decoding process itself, leveraging the observation that candidate tokens and n-gram patterns tend to recur in future sequences. Ouroboros [85] generates draft phrases from a phrase candidate pool maintained during decoding, parallelizing the drafting process and extending the length of drafts in a training-free manner. Token Recycling [89] stores candidate tokens produced during decoding in an adjacency matrix and constructs draft trees via a breadth-first search algorithm, achieving strong acceleration with minimal additional storage. For workloads with highly repetitive patterns, such as agentic pipelines, SuffixDecoding [90] caches long token sequences from prompts and previous outputs in suffix trees, adaptively adjusting speculation length based on acceptance likelihood. SAM Decoding [91] employs a suffix automaton to find the exact longest suffix match with constant-time complexity per step, and can be combined with model-based methods for further improvement. Other cache-based approaches organize token sources into hierarchical databases by temporal locality [94], construct n-gram tries from the input context [96], or maintain lightweight LRU cache tables of token n-grams [97].

D. Draft Structure and Sampling Strategies

The preceding sections focus on how draft token candidates are generated. Another important and orthogonal question is how these candidates are organized and verified. The structure used to arrange draft candidates, the sampling strategy, and the verification algorithm jointly determine the overall speedup. SpecInfer [99] introduces the *token tree*, organizing predictions from multiple speculative models into a tree for parallel verification. Sequoia [102] uses dynamic programming to find the optimal tree structure that maximizes expected accepted tokens, combined with hardware-aware tree size selection. OPT-Tree [107] similarly constructs adaptive draft trees by maximizing the expected acceptance length. GSD [108] generalizes the tree to a directed acyclic graph (DAG) that merges recurring token sequences to reduce redundancy. DDD [109] extends tree drafting with dynamic depth control. Zheng and Wang [129] propose an effective draft decoder with KL divergence alignment and a pruned candidate tree that retains only cost-effective nodes. Tree-based methods have also been extended to state-space models [123], branch parallelism [127], RL-based tree optimization [130], and block-iterative [139] and parallel [140] drafting.

Beyond tree structure, the sampling method and verification algorithm jointly determine acceptance efficiency. SpecTr [100] formulates speculative decoding through optimal transport. MCSD [101] organizes multiple candidate sequences in batches for parallel verification. SpecHub [112] simplifies the OT formulation into a compact linear program for practical use. Further analysis shows that optimal multi-draft verification decomposes into importance sampling followed by single-draft sampling [110], and that sampling without replacement outperforms with replacement [117]. Markov chain abstraction [111] reveals the fundamental limits of speculative decoding. Tree Monte Carlo [103] seeks global tree-space coupling for more accepted tokens per step. On verification, Block Verification [104] proposes provably optimal block-level verification, and Traversal Verification [122] preserves valid subsequences via leaf-to-root traversal. Connections to channel simulation [120], list-level coupling [126], convex optimization [133], hierarchical verification [135], and polybasic frameworks [131] have further enriched the theoretical landscape.

A separate line of work adaptively controls the drafting process on the fly during the runtime. DISCO [106] shows that fixed speculation lookahead is suboptimal and dynamically selects draft length based on context. SLiM [105] reduces the speculation set through fast posterior estimation. SVIP [113] uses draft model entropy to adaptively determine sequence length. BanditSpec [124] formulates hyperparameter selection as a multi-armed bandit problem with provable regret guarantees. SpecDec++ [141] casts candidate length selection as an MDP and augments the draft model with an acceptance prediction head. FailFast [142] uses a diffusion LLM drafter and adapts speculation length on the fly. TETRIS [116] optimizes batch-serving throughput by selecting promising draft tokens across requests. Speculative Speculative Decoding [137] parallelizes speculation and verification by pre-emptively preparing drafts for likely outcomes. Other methods employ draft length prediction [114], hierarchical self-verification [119], on-the-fly adaptation [128], provable drafter selection [132], RL-based optimization [136], and heterogeneity-adaptive verification [125].

Standard speculative decoding enforces strict distributional equivalence, which can be unnecessarily restrictive. A growing body of work explores relaxed verification that trades small quality deviations for larger speedups. Judge Decoding [115] trains a compact module to evaluate semantic validity of draft continuations. Fuzzy Speculative Decoding [118] accepts candidates based on target-draft divergence, enabling flexible quality-speed trade-offs. FLy [134] loosens verification by leveraging the target model's self-corrective behavior with an entropy-level gate. Alignment-augmented sampling [121] and constrained acceptance via optimization [138] have further extended the relaxed verification design space.

E. Cross-Technique Integration

Beyond accelerating standard text generation, parallel decoding has been integrated with other techniques and application domains. In constrained decoding, CDSL [143] uses the draft model to generate lookaheads that are verified by both the target model and task-specific reward functions, substantially reducing the computational cost of constrained decoding with lookahead heuristics. A particularly active direction extends speculative decoding to *multimodal generation*. DREAM [144] introduces a cross-attention mechanism to inject intermediate features from the target vision-language model (VLM) into the draft model, combined with entropy-adaptive feature selection and visual token compression. MASSV [145] transforms existing small language models into multimodal drafters by connecting the target VLM's vision encoder via a lightweight projector and applying self-distilled visual instruction tuning. ViSpec [148] introduces a vision-aware framework with a lightweight vision adaptor that compresses image tokens and augments text tokens with a global visual feature. SpecVLM [147] targets video LLMs by pruning a large fraction of video tokens during the drafting phase based on verifier attention signals, exploiting the finding that draft speculation exhibits low sensitivity to video token reduction. Spec-VLA [146] extends speculative decoding to vision-language-action models for robotic control, introducing a relaxed acceptance mechanism that leverages the relative distances represented by action tokens.

Beyond speed, the draft-then-verify mechanism can also be co-designed to carry additional functionality such as generation quality, alignment, safety, and watermarking. SCD [157] uses a single small model both to accelerate decoding and to improve generation quality through contrastive predictions. Reward-Shifted Speculative Sampling [158] instead aligns the draft model with human preferences and modifies the acceptance criterion and bonus-token distribution so that the combined system recovers the aligned distribution without tuning the target model. In a similar spirit, Speculative Safety-Aware Decoding [159] pairs the target with a small safety-enhanced model, using their agreement to estimate jailbreak risk and dynamically switching between decoding schemes to balance utility and safety while retaining acceleration. Watermarking exposes a sharper tension: a no-go theorem proves that the highest watermark strength and the highest sampling efficiency cannot be achieved simultaneously [160], and follow-up work characterizes the Pareto frontier of this trade-off, injecting pseudorandomness into draft acceptance to attain maximal watermark strength with little loss in efficiency [161].

Another emerging application is accelerating the *rollout stage* of reinforcement learning (RL) training for LLMs, where autoregressive generation of training trajectories often dominates the total training time. TLT [154] proposes an adaptive drafter that is continuously trained on idle GPUs during long-tail generation to maintain alignment with the evolving target model, combined with an adaptive rollout engine that selects suitable speculative decoding strategies for each input batch. SPEC-RL [149] reuses prior trajectory segments as speculative prefixes and extends them via draft-and-verify, avoiding redundant generation while ensuring policy consistency. FastGRPO [150] proposes concurrency-aware speculative decoding that dynamically adjusts the drafting strategy based on real-time concurrency levels, and introduces online draft learning to address distributional drift during training. ReSpec [151] addresses drafter staleness under continual actor updates by dynamically tuning configurations, evolving the drafter via knowledge distillation, and weighting updates by rollout rewards. DAS [152] targets the long-tail distribution of rollout lengths by building an adaptive drafter from recent rollouts using a suffix tree and allocating more aggressive draft budgets to long trajectories. SpecActor [153] introduces decoupled speculation for large batch sizes and a fastest-of-N method that selects and combines different draft strategies according to rollout progress. RLHFSpec [155] proposes workload-aware drafting strategy selection for the RLHF generation stage, together with efficient sample reallocation to fully utilize GPU resources. SRT [156] stores previously generated continuations in a per-prompt tree-structured cache, using it as the draft model for speculative decoding and updating trees online from ongoing rollouts.

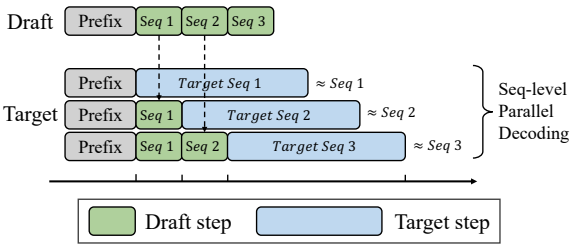


Fig. 3: Illustration of *sequence-level parallel decoding*. Sequence-level methods introduce parallelism at the granularity of reasoning steps or semantic units, enabling multiple steps to be generated or explored concurrently.

IV. Sequence-Level Parallel Decoding

Beyond the token-level parallelism discussed in Section III, parallelism can also be introduced at the level of output sequences or reasoning paths. In this section, we review *sequence-level parallel decoding* methods, which accelerate generation by parallelizing across multiple sequences, as illustrated in Figure 3. We organize them into three subcategories: multi-sequence parallel generation (Section IV-A), intra-sequence parallel generation (Section IV-B), and path exploration and test-time scaling (Section IV-C).

A. Multi-Sequence Parallel Generation

An extension of token-level parallelism is to parallelize across multiple sequences or reasoning paths. This is particularly relevant for inference-time reasoning, where methods such as tree search [324], self-consistency [325], and best-of-N sampling generate multiple candidate sequences to improve output quality, but at the cost of significant latency. FastCoT [162] provides an early exploration by using a size-varying context window to conduct parallel and autoregressive decoding simultaneously during chain-of-thought generation. SEED [163] introduces a scheduled speculative execution

framework that efficiently handles multiple iterations for thought generation and evaluation in tree-search-based reasoning, managing draft model dispatching through a rounds-scheduled strategy. SpecSearch [170] proposes a speculative search framework where a small model collaborates with a large model at both thought and token levels, employing a quality-preserving rejection mechanism to filter out thoughts whose quality falls below the target model's outputs. Speculative RAG [164] applies the speculative paradigm to retrieval-augmented generation, where a smaller specialist model generates multiple drafts from distinct document subsets in parallel, and a larger generalist model verifies them in a single pass. For multi-sample inference scenarios, Multi-Sample SD [166] exploits the intrinsic consensus across parallel reasoning paths to synthesize high-quality draft tokens without auxiliary models. STAND [173] takes a model-free approach by exploiting the redundancy in reasoning trajectories through a stochastic n-gram drafting mechanism. RSD [165] introduces reward-guided speculative decoding, which uses a process reward model to evaluate intermediate steps and dynamically decides whether to invoke the target model, prioritizing high-reward outputs. R2R [171] proposes a neural token routing method that selectively uses the large model only for critical tokens while leaving the majority of generation to the small model.

A key insight underlying many recent methods is that reasoning steps are *semantically flexible*: intermediate steps need only be logically correct, not token-identical to the target model's output. This observation enables a coarser-grained, *step-level* form of speculation that is complementary to token-level speculative decoding. SpecReason [167] exploits this flexibility by delegating simpler intermediate reasoning steps to a lightweight model, reserving the target model only to assess and potentially correct the speculated outputs. Lookahead Reasoning [172] introduces a second layer of step-level parallelism on top of token-level speculative decoding: the draft model proposes several future reasoning steps, the target model expands and verifies each in a batched pass, and a verifier accepts semantically correct steps, yielding multiplicative speedup gains. Speculative Thinking [168] takes a training-free approach where a large reasoning model guides a smaller model at the reasoning level by strategically delegating reflective steps, improving both accuracy and output conciseness. SCoT [169] conducts thought-level drafting using a lightweight model with thinking behavior alignment, then selects the best draft and corrects errors with the target model. SpecCoT [175] adopts a step-level verification strategy where the large model first establishes the reasoning direction and the small model generates multiple step candidates in parallel, which the large model verifies or replaces. Other works predict early-exit signals from the draft model to reduce reasoning overthinking [174], verify entire semantic sequences rather than individual tokens by probing model internal states [176], and perform step-level verification using model-internal attention and confidence signals [177].

B. Intra-Sequence Parallel Generation

While the methods in Section IV-A parallelize across multiple independent sequences, another direction exploits structural parallelism *within* a single output sequence. The key observation is that many LLM outputs contain semantically independent segments that can be generated concurrently. Skeleton-of-Thought [178] first demonstrates this idea by guiding the LLM to produce a skeleton of the answer, then completing each skeleton point in parallel through batched decoding. Graph of Thoughts [179] generalizes such structured decomposition to arbitrary graphs, modeling thoughts as vertices and dependencies as edges and enabling operations such as combining, distilling, and refining thoughts through feedback loops; although its primary aim is reasoning quality rather than decoding speed, the explicit dependency structure it exposes provides a natural substrate for parallel generation. Plato [180] further improves upon skeleton-based decomposition by organizing sub-problems into a dependency graph based on logical and causal relationships, decoding non-dependent nodes concurrently while preserving coherence through global context caching.

While the above methods impose parallel structure externally through prompting schemes or dedicated planning-and-decoding pipelines, a growing line of work trains models to identify and express parallelization opportunities natively. PASTA [181] introduces an annotation language that teaches LLMs to mark semantic independence directly in their responses, with an interpreter that orchestrates parallel decoding on-the-fly based on these annotations. APR [183] proposes adaptive parallel reasoning using spawn and join operations, optimizing both parent and child inference threads via end-to-end reinforcement learning without predefined reasoning structures. Multiverse [184] internalizes a MapReduce paradigm within the model itself, generating through automatic task decomposition, parallel subtask execution, and result synthesis stages, with a dedicated attention mechanism and inference engine to support natively parallel generation. SPRINT [185] introduces a post-training framework that reorganizes reasoning trajectories into structured rounds of planning and parallel execution, enabling reasoning models to dynamically identify and execute independent subtasks concurrently.

Complementary to decomposition-based methods, several works introduce architectural or algorithmic innovations to enable parallel computation within a single sequence at a more fundamental level. PCCoT [186] applies Jacobi iteration to continuous latent thought tokens, updating them iteratively in parallel rather than sequentially, improving both training and inference efficiency of continuous chain-of-thought reasoning. Tree-like attention masks [182] have been used to decode multiple parallelizable reasoning steps in a single forward pass within one sequence. Hybrid architectures that combine non-autoregressive models for generating intermediate reasoning traces with autoregressive models for producing final answers [187] and parallel loop transformers that break sequential loop dependencies through cross-loop parallelism [188] have also been explored.

C. Path Exploration and Test-Time Scaling

Test-time scaling methods such as best-of-N sampling and tree search improve output quality by exploring multiple

candidate paths; indeed, Agent Forest [189] shows that performance scales with the number of independently sampled candidates through a simple sampling-and-voting mechanism, orthogonally to more sophisticated enhancement methods. However, the computational cost of generating and evaluating these paths can be prohibitive, and a growing body of work applies speculative techniques to accelerate this exploration. Speculative Rejection [190] accelerates best-of-N inference-time alignment by early-terminating low-scoring candidates during generation, achieving the same output quality with substantially fewer computational resources. DPTS [191] introduces a dynamic parallel tree search framework for tree-of-thought reasoning that builds flexible parallelism with fine-grained cache management and filters candidates to maintain focus on promising solutions. Group Think [192] enables a single LLM to act as multiple concurrent reasoning agents with shared visibility into each other's partial progress, allowing token-level collaboration that reduces redundant reasoning. A2R [194] proposes an asymmetric two-stage framework where a lightweight explorer model generates candidate solutions in parallel and a stronger synthesizer model integrates them for refined reasoning. DTS [199] sketches a backbone tree of the reasoning space by selectively branching at decision tokens and employs early termination to prioritize shorter, more reliable trajectories.

A complementary challenge is how to efficiently allocate computation across parallel reasoning paths during test-time scaling. Parallel-R1 [195] proposes a reinforcement learning framework for training parallel thinking, using a progressive curriculum that transitions from supervised fine-tuning on easier tasks to RL exploration on harder problems, with the model learning to use parallel thinking as both an exploration strategy and a verification mechanism. Parallel Test-Time Scaling [197] extends parallel test-time scaling to latent reasoning models, introducing stochastic sampling strategies in continuous space and a latent reward model for trajectory selection. Parallel-Probe [200] introduces 2D probing that periodically elicits intermediate answers from all branches, enabling consensus-based early stopping to regulate reasoning depth and deviation-based branch pruning to dynamically adjust width. SEAT [193] uses semantic entropy to assess the quality of parallel responses and applies adaptive termination criteria for efficient multi-round collaborative inference. ATTS [196] enables asynchronous test-time scaling along both sequential and parallel axes through online calibration and ordinal classification, reducing synchronization overhead. DeepPrune [198] targets the pervasive problem of inter-trace redundancy, in which a large fraction of parallel traces yield identical answers. It trains a judge model to predict answer equivalence from partial traces and dynamically prunes redundant paths while preserving diversity.

V. Task-Level Parallel Decoding

The previous sections discuss parallelism at the token and sequence levels, where multiple tokens or reasoning paths are generated concurrently within a single model or request. At a higher granularity, parallelism can be introduced at the *task level*, where multiple agents, tool invocations, or sub-tasks are executed concurrently. This form of parallelism is particularly relevant for complex workflows such as multi-agent systems and agentic pipelines, where a task is naturally decomposed into independent sub-tasks that can proceed in parallel. In this section, we review *task-level parallel decoding* methods, as illustrated in Figure 4, organized into multi-agent parallel collaboration (Section V-A) and parallel and speculative agent execution (Section V-B).

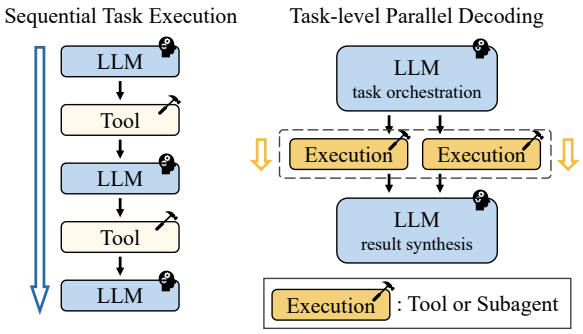


Fig. 4: Illustration of *task-level parallel decoding*. Sequential execution processes tool calls and agent actions one at a time (left). Task-level methods (right) orchestrate concurrent execution of multiple tools, sub-agents, or sub-tasks.

A. Multi-Agent Parallel Collaboration

Multi-agent systems leverage multiple LLM instances to collaboratively solve complex tasks, and their modular structure can expose opportunities for parallel execution when the underlying task graph permits. AutoGen [201] provides a foundational open-source framework that allows developers to build LLM applications via multiple customizable agents that converse with each other, supporting flexible interaction patterns through both natural language and code; it serves as generic infrastructure on which parallel multi-agent workflows can be built, rather than a parallel decoding technique itself. Building on the multi-agent paradigm, Agent Forest [189] demonstrates that LLM performance scales simply with the number of agents through a sampling-and-voting mechanism, providing an orthogonal and complementary approach to existing methods for enhancing LLMs. Mixture-of-Agents [202] proposes a layered architecture where each layer comprises multiple LLM agents, with each agent taking all outputs from the previous layer as auxiliary information, thereby leveraging the collective strengths of diverse LLMs to surpass individual model performance; while its layered aggregation primarily targets output quality, the agents within each layer are mutually independent and can generate concurrently. M1-Parallel [203] further introduces parallelism at the team level by concurrently running multiple multi-agent teams to explore distinct solution paths, using an event-driven communication model with asynchronous messaging to reduce end-to-end latency or improve task completion rates.

B. Parallel and Speculative Agent Execution

LLM-based agents frequently interact with external tools and APIs, but conventional function-calling pipelines are

typically synchronous: each call blocks the model's inference, limiting concurrent execution and increasing end-to-end latency. LLMCompiler [204] addresses this by drawing inspiration from classical compilers, introducing a function calling planner that formulates execution plans, a task fetching unit that dispatches tasks, and an executor that runs them in parallel, enabling efficient orchestration of multiple function calls. AsyncLM [208] takes a complementary approach by introducing an interrupt mechanism that asynchronously notifies the LLM when function calls return, allowing the model to continue generating while tools execute concurrently. Related approaches fuse similar tool operations under a single function call to enhance parallelism [205], adopt event-driven architectures for real-time asynchronous tool use [207], and speculatively execute predicted future tool invocations while the LLM is still generating, within a tool-aware agent-serving system [216].

The speculative execution paradigm originated in computer architecture and was later adapted to token-level decoding (Section III). A further extension applies this paradigm to entire agent actions and tool invocations. DSP [209] introduces dynamic speculative planning, an asynchronous online reinforcement learning framework that speculatively executes predicted future actions while explicitly optimizing a joint objective balancing end-to-end latency against operational cost. Speculative Actions [210] generalizes this idea to arbitrary agentic systems by using faster models to predict likely future actions and executing them in parallel, committing only when predictions match the actual model output. Other works speculate tool calls within the inference engine to reduce serving overhead [212], scale parallel tool calling width in deep research agents to reduce the number of sequential turns [213], and compress structured function calling outputs via special tokens to enable parallel generation of function names and arguments [214]. The same paradigm has since been applied to interactive planning, where a faster approximation agent proposes steps and the user may interrupt during verification [206], to search agents that may skip verification on predictable early steps [211], and to separate policies for search and visit actions [215]. Further variants predict the next tool call from the running model [217, 219] or from past trajectories [218], train a small draft on the target's actions [223], speculate tool arguments out of order [220], co-speculate actions together with their observations [221], and commit a pre-executed action chain when its first step matches [222].

VI. Architecture-Level Parallel Decoding

The methods discussed in previous sections accelerate inference while preserving the underlying autoregressive generation mechanism. A fundamentally different approach is to redesign the model architecture and training objective itself to relax the strict left-to-right token dependence, enabling inherently parallel generation. This direction encompasses a broad spectrum of techniques, including non-autoregressive models that generate all target tokens simultaneously, masked language models that iteratively predict masked positions, semi-autoregressive models that generate blocks of tokens at a time, and diffusion language models that cast generation as iterative denoising, which is typically instantiated as a mask-and-denoise process in current large-scale models. In this section, we review *architecture-level parallel decoding* methods, as illustrated in Figure 5, covering model architectures and training objectives (Section VI-A) and the associated decoding and sampling algorithms (Section VI-B).

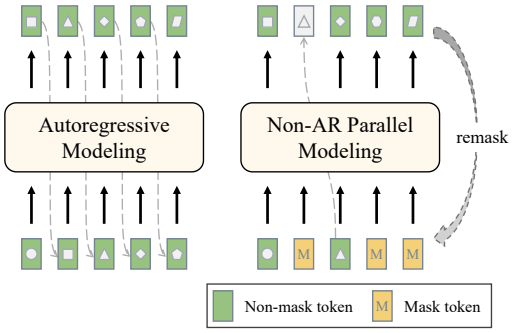


Fig. 5: Illustration of *architecture-level parallel decoding*. Autoregressive models generate tokens in a strict left-to-right order (left). Architecture-level methods (right) replace sequential generation with inherently parallel generating paradigms.

A. Model Architecture and Training

A long line of research has explored models that generate text without strict left-to-right token dependence. Non-autoregressive translation (NAT), introduced by Gu et al. [224], produces all output tokens in parallel by conditioning on fertilities as latent variables, achieving an order-of-magnitude latency reduction at a modest quality cost. Follow-up work recovered much of the lost quality through iterative decoding: Lee et al. [225] refine draft outputs over multiple denoising passes, Kaiser et al. [226] autoregressively generate a shorter sequence of discrete latent variables and decode the full output from it in parallel, and Mask-Predict [227] trains a conditional masked language model that first predicts all target tokens simultaneously and then repeatedly masks out and regenerates the least confident tokens over a fixed number of iterations. This iterative mask-and-refine mechanism anticipated the decoding style later adopted by masked diffusion language models (dLLMs).

The bridge from these early non-autoregressive models to modern dLLMs was built by *discrete diffusion*. D3PM [228] generalizes denoising diffusion to discrete state spaces via structured corruption processes, whose absorbing (masking) variant makes the connection between diffusion and mask-based generative models explicit; in parallel, Diffusion-LM [229] explored continuous diffusion over word embeddings for controllable text generation. SEDD [230] then proposed score entropy, extending score matching to discrete spaces and making discrete diffusion competitive with autoregressive models at GPT-2 scale. Subsequent work simplified and consolidated the training objective of masked (absorbing) diffusion into weighted cross-entropy forms [231, 232], and RADD [233] showed that the concrete score of absorbing diffusion reduces to time-independent conditional probabilities of

clean data. Distillation techniques further compressed many-step samplers into few-step generators [234, 235], substantially narrowing the speed gap with autoregressive models. Building on these foundations, LLaDA [21] demonstrates that a diffusion model trained from scratch under the standard pre-training and supervised fine-tuning paradigm can perform comparably to the authors' self-constructed autoregressive baselines, employing a forward masking process and a reverse generation process parameterized by a Transformer. Dream 7B [22] further validates the diffusion paradigm by initializing from a pre-trained autoregressive model and introducing context-adaptive token-level noise rescheduling, achieving strong performance in planning, arbitrary-order generation, and infilling.

Training dLLMs from scratch at scale remains costly, motivating a growing body of work on converting existing autoregressive models into diffusion models. DiffuLLaMA [236] demonstrates the feasibility of this approach by establishing connections between autoregressive and diffusion objectives and introducing a continual pre-training procedure that converts AR models into competitive diffusion models. Fast-dLLM v2 [239] substantially improves the efficiency of this conversion by combining a block diffusion mechanism with a complementary attention mask, requiring orders of magnitude fewer training tokens while matching or surpassing AR baselines. SDAR [242] proposes a synergistic paradigm that generates sequences autoregressively across blocks, while decoding all tokens within each block in parallel via discrete diffusion, demonstrating that larger models exhibit stronger robustness to block size and greater speedups. Other conversion approaches introduce next-sequence prediction for adaptive generation length [238], position-dependent masking strategies for better train-test alignment [244], and principled block-size scheduling for smoother AR-to-diffusion transition [243].

Alongside training recipes, several works have advanced the architectural foundations and scalability of diffusion language models. Block Diffusion [237] introduces a class of models that interpolate between discrete denoising diffusion and autoregressive generation, supporting flexible-length generation with KV caching and parallel token sampling within each block. E2D2 [241] proposes an encoder-decoder architecture that separates the representation of clean tokens (encoder) from the iterative denoising of corrupted tokens (lightweight decoder), accelerating both training and inference. LLaDA2.0 [245] scales the diffusion paradigm to 100B total parameters through systematic AR-to-dLLM conversion with a progressive block-level training scheme, demonstrating frontier-scale performance with parallel decoding. Large-scale MoE diffusion models [240], draft-then-refine strategies for recovering global coherence in block diffusion [246], and multi-block diffusion for inter-block parallelism [247] have further extended the design space.

B. Decoding and Sampling Algorithms

Given that diffusion language models produce predictions for all masked positions simultaneously, the inference schedule that determines which and how many tokens to commit at each denoising step plays a central role in the quality–speed trade-off. The most natural strategy is *confidence-based adaptive unmasking*, which selectively commits tokens whose prediction confidence exceeds a given criterion rather than unmasking a fixed number of tokens per step. EB-Sampler [248] observes that partially masked sequences often determine multiple token positions with near-zero entropy, and proposes an entropy-bounded unmasking procedure that dynamically unmasks multiple tokens in one forward pass while controlling the approximate error. Fast-dLLM [249] extends this idea to practical deployment by introducing a blockwise approximate KV-cache for bidirectional diffusion models and a confidence-aware strategy that selectively decodes only tokens exceeding a confidence threshold, addressing the dependency violations caused by the conditional independence assumption. Slow-Fast Sampling [250] further refines adaptive unmasking by formulating three guiding principles (certainty, convergence, and positional) that govern when and where tokens can be reliably decoded, enabling dynamic alternation between exploratory and accelerated decoding stages. Prophet [252] takes a complementary perspective by recognizing that diffusion language models often exhibit early answer convergence, where correct predictions stabilize well before the final step, and uses the confidence gap between top candidates to decide when to commit all remaining tokens at once. Along similar lines, SchED [262] introduces a progress-dependent confidence threshold based on full-span logit margins, providing stable early-exit acceleration across diverse generation tasks including long-form generation. CadLLM [263] further adapts the generation block size, step size, and confidence threshold based on the average confidence of already unmasked tokens. Exploiting the spatial structure of confidence, LocalLeap [259] identifies high-confidence anchor tokens and performs relaxed parallel decoding only within bounded neighborhoods around these anchors, committing already-determined tokens early while limiting the risk of error propagation across distant positions. From a theoretical standpoint, Cai and Li [271] provide the first formal analysis of confidence-based decoding, showing that entropy-based strategies can achieve accurate sampling with the number of iterations proportional to the data entropy, offering theoretical justification for the empirical effectiveness of these approaches.

A second line of work introduces verification and error-correction mechanisms that validate parallel drafts before committing them, or allow the decoder to revoke and revise previous decisions. WINO [251] proposes a revocable decoding framework based on a draft-and-verify paradigm: the model aggressively drafts multiple tokens in parallel and then leverages its bidirectional context to verify each prediction, re-masking suspicious tokens for further refinement, thus breaking the polarization caused by early error accumulation. FreeDave [258] extends draft-and-verification to achieve lossless parallel decoding, providing a theoretical guarantee that the decoded output is identical to the sequential one-token-per-step baseline while still reducing the number of forward passes. Saber [260] addresses the specific challenges of structured output generation by combining adaptive acceleration with a backtracking mechanism that reverts tokens whose confidence drops as surrounding context evolves. Similarly, RDD [267] detects when the denoising process fails and enables back-

tracking to earlier decoding blocks via cached model states, using confidence-guided re-masking to reinitialize uncertain positions. ReMix [268] introduces a different mechanism by inserting a continuous mixing state between the masked and decoded states: token representations are iteratively refined in continuous space to resolve inter-position conflicts before collapsing into discrete tokens, with a rejection rule reverting uncertain representations back to the masked state.

A third and more recent direction moves beyond hand-crafted heuristics by *learning* the sampling procedure itself. LSD [254] proposes learnable sampler distillation, where a student sampler with few steps is trained to align its intermediate score trajectory with that of a many-step teacher by optimizing learnable coefficients, and further learns non-uniform time schedules for better step allocation. RWS [253] takes a different approach by incorporating an external reward model to provide a global, sequence-level signal during iterative diffusion, scaling token logits according to overall sequence quality to encourage more non-autoregressive generation orders. ADJUST [255] trains a lightweight single-layer sampler on top of a frozen diffusion language model to approximate joint multi-token sampling: each expensive full-model forward pass is followed by multiple cheap passes of the sampler layer, unmasking several tokens per full-model call while staying close to the model's joint distribution. Learn2PD [256] trains a lightweight filter model in a post-training manner to predict which token positions will match their final outputs, approximating an oracle parallel decoding strategy with minimal additional training cost. dParallel [257] identifies sequential certainty convergence as the key bottleneck limiting parallelism and introduces certainty-forcing distillation that trains the model to achieve high prediction certainty on masked tokens more rapidly and in parallel; this training-time backbone modification also connects to Section VI-A. Building on the idea of learning decoding dynamics, Jazbec et al. [264] formalize masked diffusion sampling as a Markov decision process and train a lightweight policy network via reinforcement learning to map token confidences to unmasking decisions, outperforming heuristic strategies especially in the full-diffusion setting. d3LLM [266] combines pseudo-trajectory distillation during training which teaches the model which tokens can be committed early, further equipped with entropy-based multi-block decoding during inference, jointly optimizing for both accuracy and parallelism.

Finally, several methods improve sampling quality by incorporating *sequence-level structural awareness* into the token selection process. Standard confidence-based samplers select tokens independently based on local prediction certainty, which can be myopic and miss important inter-token interactions. Info-Gain Sampler [269] addresses this by balancing immediate uncertainty with the information gained over remaining masked positions, leveraging the bidirectional structure of masked diffusion models to select tokens whose unmasking maximally reduces global uncertainty. DOS [270] exploits attention matrices from transformer blocks to approximate inter-token dependencies, emphasizing information from already-decoded tokens when updating masked positions. Order-Token Search [265] jointly searches over both the generation order and token values using a likelihood estimator that scores candidate denoising actions, enabling exploration of diverse decoding trajectories rather than committing to a single fixed path. LoPA [261] concurrently explores multiple candidate token filling orders via parallel branches and selects the one with the highest potential for future parallelism, further pushing the degree of parallelism achievable in diffusion language models.

VII. System and Infrastructure

The preceding sections have reviewed parallel decoding primarily from an algorithmic perspective, introducing parallelism at the token, sequence, task, and architecture levels. Turning these algorithmic speedups into practical gains in latency and throughput, however, requires substantial system support: production serving must batch many concurrent requests, manage memory dominated by model weights and KV-caches, schedule computation across heterogeneous devices, and meet service-level objectives (SLOs) under dynamic workloads. These conditions often deviate from the assumptions under which parallel decoding algorithms are designed, such as a single request, abundant memory, and idle accelerators. In this section, we review system and infrastructure work that bridges this gap, organized into serving and scheduling (Section VII-A), which builds runtime systems for deploying parallel decoding at scale, and algorithm-infrastructure co-design (Section VII-B), which jointly optimizes decoding algorithms with the underlying infrastructure.

A. Serving and Scheduling

A first line of work adapts speculative decoding to realistic serving workloads. Early methods were developed and evaluated in the single-sequence setting, whereas batched execution is non-trivial because different sequences in a batch accept different numbers of draft tokens at each step. BASS [273] presents one of the earliest batched speculative decoding systems, addressing the challenges of multi-sequence speculative sampling and substantially improving GPU utilization in batched generation. EMS-SD [275] further removes the padding tokens that vanilla batched implementations require to align sequences within a batch, avoiding the extra computation and memory access they incur. Beyond batching, long-context serving poses its own challenge, as the KV-cache grows with sequence length and dominates memory traffic. TriForce [272] addresses this with hierarchical speculation: the target model equipped with a retrieval-based sparse KV-cache serves as an intermediate draft, which is in turn accelerated by a smaller model, enabling lossless long-sequence generation. MagicDec [278] revisits the conventional wisdom that speculative decoding benefits only small batches, showing through a careful bottleneck analysis that drafting with a sparse KV-cache yields speedups that grow with batch size for moderate-to-long sequences. Moving from decoding mechanics to serving policy, AdaSpec [283] builds a theoretical model to predict the efficiency of speculative strategies under different conditions and dynamically adjusts drafting and verification according to real-time request loads, achieving high SLO attainment.

Another line of work restructures the execution schedule of speculative decoding itself. In the standard draft-then-verify procedure, the draft and target models run alternately, so each waits idly while the other computes. PEARL [279] alleviates this mutual waiting by verifying the first draft token during drafting and continuing to draft during verification, which parallelizes the two phases and yields an adaptive draft length. DSI [274] generalizes this idea into speculation parallelism, orchestrating multiple drafter and target instances that overlap in time, and proves speedups over both speculative and autoregressive inference. Subsequent systems map such overlapped execution onto concrete hardware: Mirror-SD [285] runs complementary speculation pipelines for the draft and target models on heterogeneous accelerators, EasySpec [282] breaks inter-layer dependencies in the draft model so that its layers can run concurrently across multiple GPUs, and ALSpec [281] predicts self-attention outputs so that subsequent operations can start early on separate devices. At a lower level, Wagner et al. [277] distribute the sampling computations of speculative decoding across parallel GPU threads.

A third direction brings speculative decoding to memory-constrained platforms, where model weights must be offloaded to CPU memory or storage. SpecExec [276] observes that offloaded inference can process a large batch of tokens at nearly the cost of a single token, and exploits this by building large draft trees that the target model validates in one pass, enabling interactive inference of large models on consumer GPUs. Dovetail [280] instead assigns the two models to complementary hardware, running the draft model on the GPU and the target model on the CPU, and redesigns the drafter to reduce the communication between them. SubSpec [284] constructs the drafter directly from the offloaded portions of the target model via low-bit quantized substitute layers, obtaining a highly aligned draft model without any training while sharing the GPU-resident layers and the KV-cache.

Finally, dedicated serving infrastructure has begun to emerge for the architecture-level methods of Section VI, whose iterative denoising process exhibits memory and compute behavior fundamentally different from autoregressive decoding. dInfer [286] provides an efficient and extensible inference framework for diffusion language models, decomposing the pipeline into modular components for the model, iteration management, decoding strategy, and KV-cache management, and combining algorithmic improvements in each component with system-level optimizations. dLLM-Serve [287] further targets production serving, reducing the large transient memory footprint of diffusion decoding and interleaving requests at different phases through phase-aware scheduling. Beyond these dedicated frameworks, several dLLM decoding methods reviewed in Section VI embed substantial system contributions of their own, such as the approximate KV-cache of Fast-dLLM [249], the hierarchical caching of Fast-dLLM v2 [239], and the block-buffer pipelines of multi-block diffusion [247]. Together, these systems suggest that the serving stack, long tailored to autoregressive generation, is being rebuilt to treat parallel decoding as a first-class workload.

B. Algorithm-Infrastructure Co-Design

Another line of work jointly optimizes the decoding algorithm with the underlying hardware: decoding algorithms are tailored to exploit unique hardware characteristics, while new hardware capabilities in turn enable algorithmic designs that were previously impractical. Quantization illustrates this interplay. On the algorithm side, QSpec [289] designs the draft-verify pipeline around heterogeneous precision, drafting with fast low-precision weight-activation quantization and verifying with more accurate weight-only quantization, reusing the weights and KV-cache across the two stages so that switching between them incurs almost no cost. QuantSpec [290] further exploits the memory hierarchy of edge devices, letting the target model draft for itself with a hierarchically quantized 4-bit KV-cache and weights to directly alleviate the memory-bandwidth bottleneck of long-context inference. On the hardware side, ML-SpecQD [292] leverages the hardware-native MXFP4 format to directly cast target weights into a low-precision draft in a plug-and-play fashion without pre-training or alignment, and applies speculation recursively to accelerate the drafting stage itself.

A second thread redesigns the memory system and execution runtime around parallel decoding. Tree-structured workloads such as speculative decoding share token prefixes across branches, which conventional attention kernels fail to exploit; DeFT [288] proposes a hardware-efficient tree-attention algorithm with prefix-aware and load-balanced KV-cache partitions, eliminating redundant memory access for shared prefixes. Yggdrasil [297] instead targets the mismatch between dynamic tree speculation and static compiled runtimes, combining equal-growth tree structures with latency-aware draft selection and stage-based scheduling. Similar co-design underlies SparseSpec (Section III-B) [79], which couples sparse self-speculative drafting with a unified draft–verify scheduler, delayed verification for CPU–GPU overlap, and dynamic KV-cache management. Beyond supporting speculative decoding, the speculative principle has itself become a memory-management tool: SpeCache [293] offloads the full KV-cache to CPU memory and speculatively prefetches the entries that the next token is likely to attend to, overlapping communication with computation, and AsyncSpade [296] predicts the next-step query state so that sparse KV-cache selection can run asynchronously outside the decoding loop. Extending speculation from decoding to prefill, SpecPrefill [291] uses a lightweight model to identify important prompt tokens and feeds only those to the main model, reducing time-to-first-token in a training-free manner.

Co-design has also been examined with respect to the model architecture. Speculative decoding was long considered effective mainly for dense models, since draft tokens in a mixture-of-experts (MoE) model collectively activate more experts and thus increase the cost of verification. MoESD [294] revisits this belief and shows that at medium batch sizes sparse MoE models can in fact benefit more from speculation than dense ones, providing a theoretical model and a target-efficiency metric that expose system-level bottlenecks not captured by acceptance rates. Complementarily, utility-driven

scheduling monitors the ratio of token gains to verification cost at runtime, selectively enabling speculation and adapting the draft length per request to make speculative decoding practical for MoE serving [295]. Speculative methods that co-design the decoding rule with functional requirements such as generation quality, alignment, safety, and watermarking are algorithmic rather than infrastructural in nature, and are reviewed in Section III-E.

VIII. Hybrid Methods

The techniques reviewed in the preceding sections introduce parallelism at a single granularity, ranging from token-level drafting to architecture-level redesign and system-level support. These techniques are largely complementary rather than competing: parallelism introduced at one level does not preclude, and can often amplify, parallelism at another. A growing body of work therefore deliberately combines multiple decoding or generative paradigms within a single inference pipeline, so that their complementary strengths reinforce one another. A representative example is to pair the parallel generation capability of non-autoregressive models with the quality guarantee of autoregressive verification. In this section, we review such hybrid methods, organized into non-AR drafters for AR targets (Section VIII-A), where a parallel generator drafts blocks of tokens for an autoregressive target to verify, and speculation for other generative paradigms (Section VIII-B), which transfers the draft-then-verify principle beyond standard autoregressive language models.

A. Non-AR Drafter for AR Targets

In conventional speculative decoding, the drafter is itself autoregressive, so drafting remains a sequential process whose cost grows with the draft length and can become the new bottleneck. A natural hybrid design replaces the AR drafter with a non-AR generator that proposes an entire block of tokens in parallel or in a small number of forward passes, combining the parallelism of architecture-level methods (Section VI) with the verification safeguards of token-level speculative decoding. Parallel and CTC-based drafters of this kind have already been discussed in Section III-A [32, 33]; SpecDiff [298] instantiates the idea with a discrete diffusion model as the drafter, so that both drafting and verification are parallelized. DiffuSpec [299] makes pretrained diffusion drafters usable in a training-free manner, extracting a causally consistent left-to-right draft from the bidirectionally generated token lattice and adapting the draft length based on acceptance feedback. Since diffusion drafts are frequently rejected when the drafter is not aligned with the verifier, SpecDiff-2 [300] develops calibration techniques that match discrete diffusion drafters to autoregressive verifiers, and DEER [301] aligns the drafter through a dedicated training pipeline with single-step decoding, supporting much longer accepted drafts.

Subsequent work couples the non-AR drafter more tightly with the target model and with serving constraints. Falcon [302] enhances semi-autoregressive drafting via Coupled Sequential Glancing Distillation and a custom decoding tree. DFlash [303] conditions a lightweight block diffusion drafter on context features extracted from the target model, improving acceptance while keeping single-pass drafting. DART [304] goes further by discarding the separate diffusion model entirely, predicting logits for multiple future positions in parallel from the target model's hidden states. JetSpec [140] trains a causal parallel draft head on the target's hidden states, so that one-pass drafting still yields causally consistent draft trees rather than mutually inconsistent per-position candidates. For deployment under real workloads, DSpark [305] adopts a semi-autoregressive drafting design that couples a parallel backbone with a lightweight sequential module to restore intra-block dependencies and counter the acceptance decay of purely parallel drafts, and schedules the verification length of each request according to its confidence and the serving load. This load-aware scheduling policy also constitutes system-level co-design in the spirit of Section VII. AdaFlash [306] reduces the variance of diffusion drafters' acceptance rates across domains and token positions through on-policy distillation with a reverse-KL objective and an adaptive length head, yielding consistent gains especially under high concurrency.

B. Speculation for Other Generative Paradigms

While Section VIII-A uses non-AR models to serve AR targets, the draft-then-verify principle has also been transferred to generative paradigms beyond standard autoregressive language models. A first group of works applies speculation *within* diffusion language models to accelerate their own iterative decoding. APD [307] adaptively decides how many tokens to sample in parallel by defining a multiplicative mixture of the dLLM's marginal probabilities and the joint probability of sequences under a small auxiliary autoregressive model, which provides inter-token dependency constraints rather than acting as a standard accept/reject verifier. Other work removes the auxiliary model altogether: SSD [308] lets the dLLM act as its own drafter and verifier through hierarchical verification trees, keeping the output identical to stepwise decoding; Spiffy [309] organizes self-drafted states into directed draft graphs that match the blockwise, bidirectional nature of dLLM generation and are calibrated offline to maximize acceptance. ODB-dLLM [310] further embeds a dLLM-specific speculative scheme into a whole-pipeline acceleration framework that also adapts the response length during prefill, balancing the heterogeneous costs of the two phases.

Speculation has also been unified with model cascades, an efficiency paradigm that interleaves models of different sizes through a deferral rule rather than parallel verification. CS Drafting [311] applies the cascade idea inside the drafting stage, using a vertical cascade to eliminate autoregressive generation by neural drafters and a horizontal cascade to allocate the drafting budget according to token importance. Faster Cascades [312] take the complementary route, implementing the cascade deferral rule through speculative execution and thereby combining the favorable cost-quality trade-offs of cascades with the quality guarantees of speculative decoding.

Finally, the draft-then-verify principle extends beyond language generation altogether; while the following works fall outside language models in the strict sense, we briefly include

them as evidence of the generality of the speculation principle. For autoregressive text-to-image models, SJD [313] replaces the deterministic convergence criterion of Jacobi iteration with a probabilistic acceptance rule, making training-free parallel decoding compatible with sampling-based generation, and LANTERN [314] tackles the token selection ambiguity of visual AR models by relaxing the acceptance condition with interchangeable tokens in latent space under a bounded distributional deviation. Speculative sampling has likewise been generalized from discrete sequences to the continuous Markov chains of diffusion models, with drafting strategies, including a training-free variant, that still produce exact samples from the target [315], and a complementary study proves that suitably reparametrized DDPM increments are exchangeable, enabling auto-speculative parallel sampling with provable speedup and no draft model at all [316]. Collectively, these results establish speculation as a general principle for parallelizing sequential generation processes.

IX. Benchmarks and Evaluation

This section reviews related evaluation resources, including surveys, benchmark suites, and diagnostic studies.

Related surveys. With the growing interest in parallel decoding, several surveys have reviewed parts of this landscape. An early survey of speculative decoding [317] formalizes the draft-then-verify paradigm, organizes existing methods around drafter selection and verification strategies, and compares leading approaches under third-party testing environments. More recent surveys examine parallel text generation, categorizing techniques into AR-based and non-AR-based paradigms [318], and parallel reasoning, which concurrently explores multiple lines of thought before converging on a final answer [319]. Each of these surveys, however, targets a single facet of the field: token-level speculation, text-generation algorithms, or reasoning-level parallelism. In contrast, our survey aims to provide a broader unified view, organizing all of these threads under a taxonomy based on the *granularity* at which parallelism is introduced, spanning the token, sequence, task, architecture, and system levels, and further covering hybrid methods and evaluation resources.

Benchmark suites. Unified benchmarks have been developed to support fair and reproducible comparison. Spec-Bench [320] evaluates open-source speculative decoding methods on the same tasks, devices, and testing conditions, providing the community with realistic speedup expectations across application scenarios. Extending benchmarking to emerging workloads, a recent benchmark for test-time scaling [321] compares model-based, training-based, and n-gram-based speculative decoding under consistent protocols, and finds that n-gram-based methods are particularly effective on the repetitive reasoning traces that test-time scaling produces.

Diagnostic studies. Beyond comparing individual methods, diagnostic studies assess whether the efficiency promised by parallel decoding is realized in practice. For diffusion language models, a systematic examination of evaluation practices [322] shows that, under carefully controlled measurements, current open-source diffusion models generally attain lower throughput than autoregressive counterparts, and that popular acceleration techniques bring benefits mainly at small batch sizes, underscoring the need for rigorous measurement protocols. ParallelBench [323] analyzes the speed-quality trade-off from an information-theoretic perspective: under the conditional-independence approximation adopted by current dLLM parallel decoders, jointly decoded tokens fail to capture inter-token dependencies. Based on this analysis, it constructs tasks with strong token dependencies that are easy for autoregressive models yet challenging for dLLMs under parallel decoding, and demonstrates that current dLLM decoding strategies cannot adapt their degree of parallelism to task difficulty, leading to severe quality degradation when parallelism is pushed aggressively. These studies delineate the conditions under which parallel decoding is genuinely beneficial, and motivate evaluation protocols that report speed and quality jointly under realistic workloads.

X. Experimental Comparison

The preceding sections have reviewed parallel decoding from a conceptual perspective. Because published speedups are often measured under mismatched models, tasks, and serving stacks, fair comparison remains difficult. We therefore evaluate representative methods from our taxonomy under an identical experimental setup. Our goal is not to declare a single winner, but to characterize how much acceleration each paradigm delivers, at what cost in quality, and on which tasks it is preferable.

Selected Methods. We compare standard AR decoding with representative methods drawn from four layers of our taxonomy: token-level, sequence-level, architecture-level, and hybrid methods. Unless otherwise noted, methods that draft for an autoregressive target adopt Qwen3-8B [8] as the target model. We additionally include Qwen2.5-7B [326] as an AR baseline for size-matched architecture-level comparisons. At the token level, we include (i) NGram, a training-free n-gram drafting baseline; and (ii) EAGLE-3 [68], a lightweight autoregressive draft head with multi-layer feature fusion and tree verification. At the sequence level, we include (iii) Lookahead Reasoning [172], an application-layer step-level method that drafts reasoning steps with a small model and accepts them via an LLM-as-judge. At the architecture level, we include (iv) LLaDA [21], a from-scratch masked-diffusion backbone evaluated as a serial full-attention baseline; (v) Dream [22], a discrete-diffusion backbone initialized from Qwen2.5-7B; (vi) Fast-dLLM v2 [239], adapted from Qwen2.5-7B-Instruct into a block-diffusion model with hierarchical caching; (vii) SDAR [242], converted from Qwen3-8B into a block-diffusion model; and (viii) d3LLM-LLaDA and (ix) d3LLM-Dream [266], which retain the full-attention backbones of LLaDA and Dream but add pseudo-trajectory distillation and entropy-based multi-block decoding mechanisms. As hybrid methods, we include (x) DFlash [303], a one-step block-diffusion parallel drafter; (xi) JetSpec [140], which proposes a candidate tree in one forward pass under branch-wise causal conditioning; (xii) DSpark [305], a semi-autoregressive

TABLE I: Speedup and average acceptance length τ of speculative decoding methods on seven benchmarks (Qwen3-8B).

	MATH						CODE				CHAT						
	GSM8K		MATH-500		AIME25		HumanEval		MBPP		MT-Bench		Arena-Hard		*Avg.*		
Method	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	AUP
GPU: Nvidia A800																	
Lookahead Reasoning	1.29×	2.47	1.29×	2.79	1.32×	2.71	1.10×	1.49	1.05×	1.53	1.07×	1.71	1.01×	1.53	1.16×	2.03	159.0
NGram	1.75×	2.08	2.28×	2.68	2.31×	2.76	1.67×	1.97	1.83×	2.17	1.19×	1.42	1.34×	1.56	1.77×	2.09	174.3
EAGLE-3	2.09×	3.09	2.18×	3.15	2.21×	3.28	2.19×	3.23	2.03×	3.00	1.75×	2.62	1.74×	2.57	2.03×	2.99	246.0
DSpark	4.49×	6.42	4.78×	6.56	4.46×	6.20	4.37×	6.18	3.82×	5.48	2.83×	3.93	2.90×	4.04	3.95×	5.55	489.2
DFlash	4.27×	5.98	6.17×	8.41	5.95×	7.98	5.15×	7.27	4.09×	5.75	2.45×	3.32	2.55×	3.43	4.38×	6.02	542.2
AdaFlash	5.08×	7.30	6.22×	8.84	5.86×	8.28	5.30×	7.56	4.13×	5.97	2.43×	3.45	2.55×	3.60	4.51×	6.43	587.1
JetSpec	4.62×	6.36	5.83×	7.81	5.72×	7.68	4.64×	6.36	4.29×	5.68	3.16×	4.20	3.25×	4.27	4.50×	6.05	516.7
GPU: Nvidia H800																	
Lookahead Reasoning	1.31×	2.51	1.29×	2.77	1.35×	2.75	1.09×	1.48	1.07×	1.54	1.08×	1.72	1.00×	1.52	1.17×	2.04	159.5
NGram	1.58×	2.08	2.04×	2.63	2.31×	2.96	1.51×	1.96	1.68×	2.18	1.10×	1.41	1.22×	1.57	1.63×	2.11	173.8
EAGLE-3	1.96×	3.09	2.02×	3.14	2.07×	3.20	2.06×	3.24	1.90×	3.00	1.69×	2.63	1.66×	2.57	1.91×	2.98	245.3
DSpark	4.58×	6.41	4.82×	6.52	4.67×	6.25	4.47×	6.15	3.92×	5.46	2.86×	3.90	2.99×	4.04	4.04×	5.53	487.1
DFlash	4.38×	5.97	6.27×	8.31	6.05×	7.94	5.37×	7.27	4.21×	5.74	2.48×	3.29	2.59×	3.41	4.48×	5.99	539.4
AdaFlash	5.17×	7.48	6.42×	8.92	6.22×	8.55	5.28×	7.58	4.22×	6.07	2.48×	3.46	2.62×	3.60	4.62×	6.52	595.8
JetSpec	4.73×	6.26	5.69×	7.70	5.36×	7.41	4.80×	6.37	4.29×	5.68	3.31×	4.15	3.29×	4.28	4.50×	5.98	515.1
GPU: Nvidia B200																	
Lookahead Reasoning	1.24×	2.48	1.24×	2.67	1.19×	2.28	1.08×	1.52	1.03×	1.55	1.04×	1.78	1.01×	1.53	1.11×	1.97	157.9
NGram	1.43×	2.08	1.80×	2.65	1.99×	2.85	1.36×	1.98	1.49×	2.17	0.99×	1.42	1.09×	1.57	1.45×	2.10	174.5
EAGLE-3	1.68×	3.09	1.73×	3.15	1.77×	3.17	1.76×	3.22	1.63×	2.99	1.44×	2.63	1.42×	2.57	1.64×	2.97	248.0
DSpark	3.75×	6.40	3.92×	6.52	3.83×	6.16	3.66×	6.20	3.19×	5.43	2.38×	3.94	2.47×	4.05	3.31×	5.53	486.8
DFlash	3.59×	5.96	5.10×	8.35	5.08×	8.10	4.31×	7.24	3.40×	5.72	2.03×	3.29	2.14×	3.43	3.66×	6.01	540.8
AdaFlash	4.38×	7.33	5.15×	8.83	4.83×	8.28	4.38×	7.53	3.53×	5.98	2.10×	3.45	2.21×	3.60	3.80×	6.43	587.6
JetSpec	5.48×	7.48	7.73×	10.54	7.52×	10.10	5.60×	7.67	5.18×	6.59	3.63×	4.51	3.68×	4.64	5.55×	7.36	636.2

parallel drafter that couples a sequential head with confidence-scheduled verification; and (xiii) AdaFlash [306], which extends DFlash-style diffusion drafting with a resverse-KL objective and an adaptive length head.

Benchmark Datasets. We evaluate on seven benchmarks spanning three domains: mathematical reasoning, including GSM8K [327], MATH-500 [328], and AIME25 [329]; code generation, including HumanEval [330] and MBPP [331]; and open-ended dialogue, including MT-Bench [332] and Arena-Hard [333]. For GSM8K, MATH-500, and AIME25, we follow an lm-evaluation-style prompt protocol and set the maximum output lengths to 1,024, 4,096, and 4,096 tokens, respectively. For the remaining datasets, and for all dLLM runs, the maximum generation budget is 512 tokens.

Metrics. For speculative decoding, we report end-to-end speedup relative to the matching AR baseline, average acceptance length τ, and AUP. Accuracy is the solve rate on mathematical benchmarks and EvalPlus pass@1 on code benchmarks. For dLLMs, we report tokens per forward (TPF), end-to-end speedup, and AUP, with accuracy shown along the curves in Figure 7. Speedup is measured against a size-matched AR baseline on the same serving stack: LLaDA, Dream, Fast-dLLM v2, and d3LLM are normalized to Qwen2.5-7B, while SDAR is normalized to Qwen3-8B.

Besides, we adopt Accuracy Under Parallelism (AUP) [266] to jointly evaluate accuracy and parallelism across the two paradigms. Parallelism is measured by TPF for dLLMs and average accepted length τ for speculative decoding. Specifically, for the m retained points, we compute

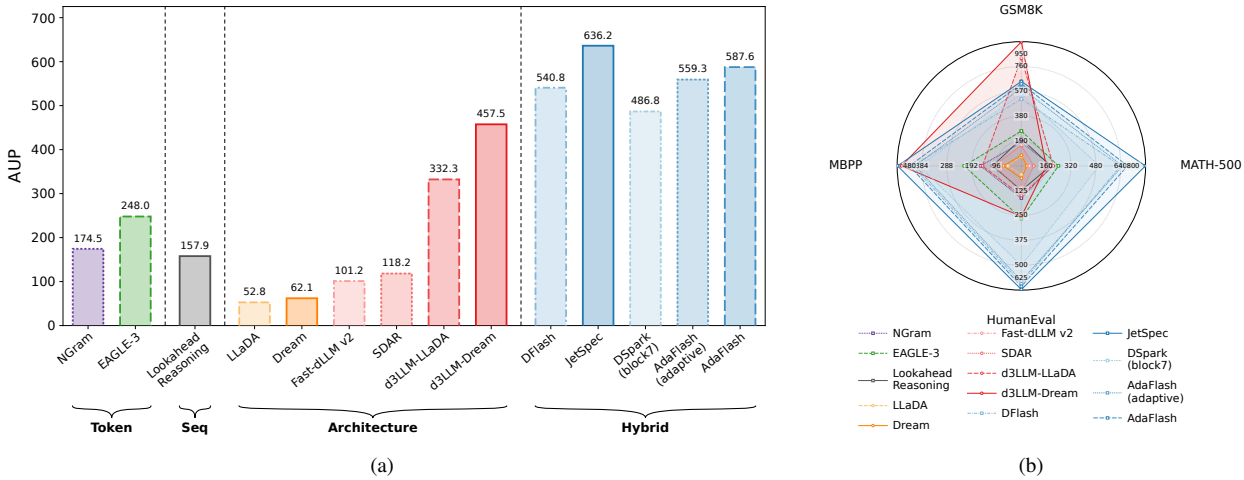
$$\mathrm{AUP} = \rho_1 y_1 + \sum_{i=2}^{m} \frac{\rho_i - \rho_{i-1}}{2} \cdot \big[y_i W(y_i) + y_{i-1} W(y_{i-1})\big],$$

where $W(y) = \min\left\{\exp\left[-\alpha\left(1 - \frac{y}{y_{\max}}\right)\right], 1\right\}$ is the penalty function, with $\alpha = 3$. Thus, the initial rectangle $\rho_1 y_1$ is unweighted, and subsequent intervals use the trapezoidal rule on $yW(y)$. For lossless speculative decoding, we use the product extension $\mathrm{AUP} = \tau a$, where a is the accuracy in percent. For Lookahead Reasoning, we use its measured accuracy and step-level accepted length τ_{step}, defined as the average number of reasoning steps committed per speculation cycle, including the terminal step. We report mean AUP over GSM8K, MATH-500, HumanEval, and MBPP, with code accuracy averaged across the EvalPlus base and plus settings.

Implementation Details. We evaluate on different hardware platforms, including Nvidia A800, H800, and B200 under the same setup and report each GPU independently. Speculative methods use SGLang Spec V2 [17], except Lookahead Reasoning, which runs on stock vLLM with a three-GPU target–draft–judge placement, and JetSpec, which uses its native engine. Each speedup is normalized to the matching AR baseline on the same stack. We follow official default hyperparameters where possible. EAGLE-3 uses a third-party AngelSlim Qwen3-8B draft head with a draft tree of depth 5, branching factor 4, and 16 draft tokens; speedup with this draft may be lower than that of the official Llama-trained EAGLE-3 drafter. Lookahead Reasoning uses Qwen3-0.6B as the step drafter and Qwen2.5-7B-Instruct as the judge, with maximum lookahead depth 4, width 1, and 100 tokens per step. DFlash, DSpark, and AdaFlash use block sizes 16, 7, and 16, respectively. The smaller block of DSpark is inherited from its officially released drafter and largely accounts for its lower τ and speedup relative to DFlash and AdaFlash. JetSpec uses a causal draft tree of width 7 and depth 20, with node budgets of 64 on A800 and H800 and 128 on B200. For AdaFlash, we

TABLE II: Average accept rate, draft length k, and acceptance length τ of speculative decoding methods on seven benchmarks (Qwen3-8B). AdaFlash (Adaptive) enables the adaptive length head in Qian et al. [306].

		MATH						CODE				CHAT					
		GSM8K		MATH-500		AIME25		HumanEval		MBPP		MT-Bench		Arena-Hard		*Avg.*	
Method	k	AR	τ	AR	τ	AR	τ	AR	τ	AR	τ	AR	τ	AR	τ	AR	τ
Lookahead Reasoning	$\leqslant$4	0.693	2.49	0.682	2.74	0.643	2.58	0.422	1.50	0.465	1.54	0.502	1.74	0.401	1.53	0.544	2.02
NGram	15	0.072	2.08	0.110	2.65	0.124	2.86	0.065	1.97	0.080	2.17	0.028	1.42	0.038	1.57	0.074	2.10
EAGLE-3	15	0.139	3.09	0.143	3.15	0.148	3.22	0.148	3.23	0.132	3.00	0.108	2.63	0.105	2.57	0.132	2.98
DSpark	7	0.776	6.41	0.791	6.53	0.744	6.20	0.743	6.18	0.642	5.46	0.419	3.92	0.437	4.04	0.651	5.54
DFlash	15	0.332	5.97	0.491	8.36	0.468	8.01	0.419	7.26	0.317	5.74	0.154	3.30	0.162	3.42	0.335	6.01
AdaFlash	15	0.425	7.37	0.524	8.86	0.491	8.37	0.438	7.56	0.334	6.01	0.164	3.45	0.174	3.60	0.364	6.46
AdaFlash (Adaptive)	$\leqslant$15	0.682	7.00	0.754	8.44	0.761	8.01	0.676	7.25	0.587	5.70	0.599	3.12	0.595	3.23	0.665	6.11
JetSpec (64)	63	0.085	6.31	0.107	7.75	0.104	7.54	0.085	6.37	0.074	5.68	0.051	4.18	0.052	4.28	0.080	6.02
JetSpec (128)	127	0.051	7.48	0.075	10.54	0.072	10.10	0.053	7.67	0.044	6.59	0.028	4.51	0.029	4.64	0.050	7.36



(a) (b)

Fig. 6: AUP overview by taxonomy level. (a) Average AUP over GSM8K, MATH-500, HumanEval, and MBPP, grouped into token-, sequence-, architecture-, and hybrid-level methods. (b) Radar charts of AUP scores on different datasets.

evaluate with a fixed-verification-length version with $k = 15$. dLLM methods run in SGLang. d3LLM uses block size 32 with KV-cache refresh delay 2. In the tables, Avg. is the mean over the reported datasets.

Results and Analysis. As shown in Table I, parallel drafting substantially outperforms the autoregressive draft head under matched AR baselines. EAGLE-3 yields only about $2\times$ speedup with $\tau \approx 3$. Lookahead Reasoning yields only about 1.1–$1.3\times$ against its own vLLM AR baseline; gains are larger on math than on code or chat, and its τ_{step} should not be read as a token-level acceptance length. In contrast, DFlash, JetSpec, DSpark, and AdaFlash typically reach 4–$6\times$ on mathematical and coding tasks, with τ in the 6–9 range. The gap arises because these methods propose many candidates in one draft forward, rather than extending an autoregressive draft chain. Among SGLang methods, AdaFlash with fixed verification length is strongest overall. For example, on A800 it attains $5.08\times$ speedup and $\tau = 7.30$ on GSM8K. On B200, DFlash reaches $2.03\times$ and $2.14\times$ on MT-Bench and Arena-Hard, respectively, while JetSpec reaches $3.63\times$ and $3.68\times$. The average-speedup ranking depends on the platform and configuration: AdaFlash leads on A800 and H800, while JetSpec leads on B200, where its node budget is larger. We therefore emphasize the Avg. and AUP columns below.

Table II reports accept rate together with draft length k and accepted length τ. Here k denotes the draft length: the number of candidates proposed in each speculation cycle, counted in tokens for standard speculative decoding and in reasoning steps for Lookahead Reasoning. For Lookahead Reasoning, the maximum lookahead depth is set to 4. Under asynchronous execution, the effective draft length k may be smaller due to early rejection, unfinished drafting, or EOS. NGram proposes long drafts but accepts only about 0.07 of them, bringing little acceleration. EAGLE-3 uses k=15 with accept rate about 0.13, giving $\tau \approx 3$. Among parallel drafters, DFlash and AdaFlash with fixed verification use k=15 with moderate accept rates and correspondingly larger τ. JetSpec builds a large draft tree: with k=63 the accept rate is about 0.08 and $\tau \approx 6.0$, while the larger k=127 budget further lowers the accept rate to about 0.05 but raises τ to about 7.4. DSpark attains a high accept rate of about 0.65, with draft length capped at k=7. AdaFlash with adaptive verification reports a higher accept rate mainly by shortening the verification window, rather than by improving draft quality. Under batch size one, it does not outperform the fixed ablation. Gains remain domain-dependent and are stronger on math and coding tasks than on dialogue.

Turning to dLLMs in Table III, serial LLaDA and Dream stay $\text{TPF} = 1$ and fall well below matched AR throughput. This reflects the cost of iterative masked decoding without KV-cache. Fast-dLLM v2 and SDAR occupy a moderate regime, with $\text{TPF} \approx 1.3$–2.7 and about 1–$2\times$ speedup, while preserving higher code accuracy than the full-attention

TABLE III: dLLM speedup and tokens per forward (TPF) at the selected operating points, with the AUP metric integrated over the retained accuracy–parallelism curves. Code AUP uses EvalPlus base pass@1.

	MATH						CODE						CHAT						
	GSM8K			MATH-500			HumanEval			MBPP			MT-Bench		Arena-Hard		*Avg.*		
Method	Speedup	TPF	AUP	Speedup	TPF	AUP	Speedup	TPF	AUP	Speedup	TPF	AUP	Speedup	TPF	Speedup	TPF	Speedup	TPF	AUP
GPU: Nvidia A800																			
LLaDA	0.16×	1.00	74.8	0.16×	1.00	37.2	0.16×	1.00	42.7	0.17×	1.00	52.1	0.18×	1.00	0.16×	1.00	0.16×	1.00	51.7
Dream	0.20×	1.00	81.1	0.17×	1.00	37.0	0.17×	1.00	63.4	0.19×	1.00	68.3	0.20×	1.00	0.19×	1.00	0.19×	1.00	62.5
Fast-dLLM v2	1.87×	2.30	149.0	2.16×	2.67	53.1	1.70×	2.10	101.5	1.21×	1.47	78.2	1.12×	1.33	1.14×	1.39	1.53×	1.88	95.5
SDAR	1.12×	1.58	144.3	1.36×	1.79	77.9	1.01×	1.52	123.8	1.03×	1.35	106.5	0.90×	1.20	0.90×	1.20	1.05×	1.44	113.1
d3LLM-LLaDA	2.83×	16.86	813.6	1.46×	8.60	222.2	1.80×	10.50	154.0	1.86×	10.30	187.2	0.71×	3.60	0.66×	3.90	1.55×	8.96	344.3
d3LLM-Dream	3.11×	15.27	955.1	1.98×	9.21	210.0	1.94×	9.38	276.0	2.32×	10.11	509.6	1.00×	3.66	0.71×	3.35	1.84×	8.50	487.7
GPU: Nvidia H800																			
LLaDA	0.32×	1.00	75.1	0.31×	1.00	38.5	0.30×	1.00	43.1	0.31×	1.00	53.3	0.32×	1.00	0.30×	1.00	0.31×	1.00	52.3
Dream	0.36×	1.00	82.5	0.35×	1.00	37.4	0.34×	1.00	64.1	0.36×	1.00	69.4	0.37×	1.00	0.34×	1.00	0.35×	1.00	63.4
Fast-dLLM v2	1.80×	2.29	149.5	1.71×	2.77	55.8	1.63×	2.09	102.2	1.17×	1.45	79.2	1.07×	1.34	1.12×	1.39	1.42×	1.78	96.6
SDAR	1.10×	1.74	158.3	1.21×	1.79	75.0	0.91×	1.52	125.9	0.93×	1.35	106.7	0.92×	1.36	0.91×	1.37	1.00×	1.52	116.4
d3LLM-LLaDA	5.46×	16.79	826.3	2.74×	8.46	207.3	3.13×	10.48	149.0	3.54×	10.41	196.1	1.29×	3.53	1.23×	3.96	2.90×	8.94	344.7
d3LLM-Dream	6.02×	15.30	955.2	3.25×	8.94	195.2	3.47×	9.44	263.7	4.66×	10.16	492.7	1.81×	3.58	1.28×	3.37	3.42×	8.30	476.7
GPU: Nvidia B200																			
LLaDA	0.30×	1.00	75.0	0.30×	1.00	40.8	0.27×	1.00	43.3	0.29×	1.00	52.1	0.31×	1.00	0.28×	1.00	0.29×	1.00	52.8
Dream	0.30×	1.00	81.6	0.31×	1.00	35.4	0.28×	1.00	62.2	0.30×	1.00	69.0	0.31×	1.00	0.30×	1.00	0.30×	1.00	62.1
Fast-dLLM v2	1.59×	2.47	159.6	1.50×	2.30	52.4	1.43×	2.24	108.8	1.00×	1.53	83.9	0.91×	1.37	0.97×	1.46	1.24×	1.90	101.2
SDAR	0.84×	1.74	157.4	0.91×	1.80	79.9	0.70×	1.54	129.9	0.71×	1.35	105.7	0.69×	1.35	0.69×	1.36	0.75×	1.52	118.2
d3LLM-LLaDA	5.13×	16.73	822.9	2.66×	8.53	206.3	2.98×	10.50	149.8	2.26×	7.09	150.2	1.27×	3.58	1.17×	3.89	2.58×	8.39	332.3
d3LLM-Dream	5.09×	15.17	949.8	2.85×	7.91	162.7	3.11×	9.42	254.2	3.95×	9.77	463.1	1.66×	3.67	1.11×	3.30	2.96×	8.21	457.5

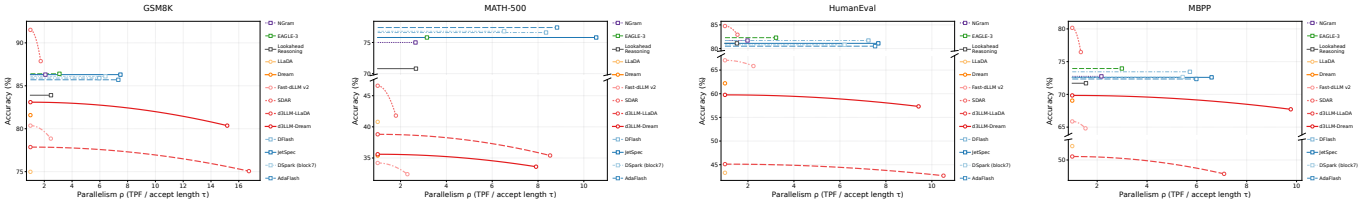


Fig. 7: Accuracy–parallelism frontiers. The horizontal axis is parallelism (τ for speculative decoding, TPF for dLLMs).

d3LLM variants. Under the same accuracy tolerance, d3LLM-LLaDA and d3LLM-Dream reach much higher parallelism. On GSM8K, their endpoint TPF is about 15–17, corresponding to about 3–6× speedup, and they achieve the highest dLLM AUP. The same results also highlight a distinctive flexibility of dLLMs: on easier problems they can adaptively raise TPF by a large margin and thereby improve throughput, whereas on harder tasks they stay more conservative to protect accuracy.

We next compare AUP across taxonomy levels on B200. Hybrid methods obtain the highest group mean, led by JetSpec and AdaFlash (fixed), as shown in Figure 6. The architecture-level group spans a wide range and falls below the token-level mean. At the token level, EAGLE-3 exceeds NGram; both remain below the evaluated hybrid methods. Within architecture-level methods, serial LLaDA and Dream stay near the bottom, Fast-dLLM v2 and SDAR occupy a middle band, and d3LLM-LLaDA and d3LLM-Dream lead this level. The accuracy–parallelism frontiers in Figure 7 make the underlying difference explicit. Lossless speculation yields nearly horizontal frontiers, so AUP mainly tracks the attained τ. dLLMs instead expose a flexible trade-off: they raise TPF aggressively when the task allows, and retreat when accuracy would otherwise degrade.

To summarize, among the evaluated paradigms, speculative decoding, especially hybrid methods, delivers the strongest lossless acceleration, typically reaching 4–6× speedup on mathematical and coding tasks. dLLMs offer an advantage of efficiency at the cost of lower accuracy. Taken together, these results confirm that parallel decoding is a promising and practical direction for accelerating LLM inference.

XI. Conclusion

This survey presented a comprehensive review of parallel decoding methods for large language models, organized by the granularity at which parallelism is introduced. Specifically, we categorized existing work into seven classes: (i) *token-level* methods that generate and verify multiple candidate tokens within a single decoding step; (ii) *sequence-level* methods that parallelize across reasoning paths and structurally independent segments; (iii) *task-level* methods that enable concurrent agent execution and tool invocation; (iv) *architecture-level* methods that replace autoregressive generation with inherently parallel paradigms such as discrete diffusion language models; (v) *system-level* optimizations that deploy parallel decoding in production serving infrastructures; (vi) *hybrid* methods that combine techniques across multiple levels; and (vii) *benchmarks and evaluation* resources tailored to parallel decoding. We further conducted a unified experimental comparison of representative methods. For future work, several important

challenges remain open. Extending parallel decoding to long-context and long-form generation, where maintaining both quality and speedup as the output length scales to tens of thousands of tokens, is a largely unexplored problem. Beyond text, adapting these techniques to multimodal generation, including image, video, and speech, presents new opportunities. More broadly, as LLMs are increasingly deployed in agentic workflows, world models, and embodied intelligence, the scope of parallel decoding can be extended to these scenarios where sequential dependencies arise at different levels. We hope this survey serves as a useful reference for researchers and inspires continued progress in efficient inference for LLMs.

REFERENCES

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 1877–1901.

[2] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 27 730–27 744.

[3] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *ArXiv preprint*, vol. arXiv:2307.09288, 2023.

[4] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *ArXiv preprint*, vol. arXiv:2303.08774, 2023.

[5] S. Huang, T. Cheng, J. K. Liu, W. Xu, J. Hao, L. Song, Y. Xu, J. Yang, J. Liu *et al.*, "Opencoder: The open cookbook for top-tier code large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 33 167–33 193.

[6] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[7] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *ArXiv preprint*, vol. arXiv:2412.19437, 2024.

[8] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu *et al.*, "Qwen3 technical report," *ArXiv preprint*, vol. arXiv:2505.09388, 2025.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 24 824–24 837.

[10] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "Deepseek-r1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, 2025.

[11] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.

[12] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *ArXiv preprint*, vol. arXiv:2210.17323, 2022.

[13] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.

[14] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 16 344–16 359.

[15] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng, "Native sparse attention: Hardware-aligned and natively trainable sparse attention," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 23 078–23 097.

[16] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of the 6th Conference on Machine Learning and Systems (MLSys)*, 2023, pp. 606–624.

[17] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 62 557–62 583.

[18] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 19 274–19 286.

[19] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *ArXiv preprint*, vol. arXiv:2302.01318, 2023.

[20] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 5209–5235.

[21] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li, "Large language diffusion

models," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 50 608–50 646.

[22] J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong, "Dream 7b: Diffusion large language models," *ArXiv preprint*, vol. arXiv:2508.15487, 2025.

[23] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018, pp. 10 107–10 116.

[24] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui, "Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 3909–3925.

[25] S. Kim, K. Mangalam, S. Moon, J. Malik, M. W. Mahoney, A. Gholami, and K. Keutzer, "Speculative decoding with big little decoder," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 39 236–39 256.

[26] Y. Zhou, K. Lyu, A. S. Rawat, A. K. Menon, A. Rostamizadeh, S. Kumar, J.-F. Kagy, and R. Agarwal, "Distillspec: Improving speculative decoding via knowledge distillation," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[27] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang, "Online speculative decoding," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 31 131–31 146.

[28] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding speculative decoding," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 6460–6473.

[29] E. Yi, T. Kim, H. Jeung, D.-S. Chang, and S.-Y. Yun, "Towards fast multilingual LLM inference: Speculative decoding and specialized drafters," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 10 789–10 802.

[30] L. Zhang, X. Wang, Y. Huang, and R. Xu, "Learning harmonized representations for speculative sampling," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[31] S. Ouyang, S. Wang, M. Jiang, M. Zhong, D. Yu, J. Han, and Y. Shen, "Temperature-centric investigation of speculative decoding with knowledge distillation," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 13 125–13 137.

[32] Z. Xiao, H. Zhang, T. Ge, S. Ouyang, V. Ordonez, and D. Yu, "Parallelspec: Parallel drafter for efficient speculative decoding," *ArXiv preprint*, vol. arXiv:2410.05589, 2024.

[33] Z. Wen, S. Gui, and Y. Feng, "Speculative decoding with ctc-based draft model for llm inference acceleration," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 92 082–92 100.

[34] N. Timor, J. Mamou, D. Korat, M. Berchansky, G. Jain, O. Pereg, M. Wasserblat, and D. Harel, "Accelerating LLM inference with lossless speculative decoding algorithms for heterogeneous vocabularies," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 59 598–59 620.

[35] J. Fu, Y. Jiang, J. Chen, J. Fan, X. Geng, and X. Yang, "Fast large language model collaborative decoding via speculation," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 17 764–17 782.

[36] S. Hu, J. Li, X. Xie, Z. Lu, K.-c. Toh, and P. Zhou, "Griffin: Effective token alignment for faster speculative decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 125 648–125 673.

[37] Y. Weng, D. Mei, H. Qiu, X. Chen, L. Liu, J. Tian, and Z. Shi, "CORAL: Learning consistent representations across multi-step training with lighter speculative drafter," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 5580–5593.

[38] W. Zhao, T. Pan, X. Han, Y. Zhang, A. Sun, Y. Huang, K. Zhang, W. Zhao, Y. Li, J. Zhou, H. Zhou, J. Wang, Z. Liu, and M. Sun, "FR-spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 3909–3921.

[39] F. Hong, R. Raju, J. L. Li, B. Li, U. Thakker, A. Ravichandran, S. Jain, and C. Hu, "Training domain draft models for speculative decoding: Best practices and insights," in *First Workshop on Scalable Optimization for Efficient and Adaptive Foundation Models*, 2025.

[40] D. Ge, J. Gao, Q. Jiang, Y. Feng, and W. Ji, "Automatic task detection and heterogeneous llm speculative decoding," *ArXiv preprint*, vol. arXiv:2505.08600, 2025.

[41] D. Choi, S. Oh, S. Dingliwal, J. Tack, K. Kim, W. Song, S. Kim, I. Han, J. Shin, A. Galstyan, S. Katiyar, and S. B. Bodapati, "Mamba drafters for speculative decoding," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 21 404–21 418.

[42] R. Kinattinkara Ramakrishnan, Z. Yuan, S. Zhuo, C. Feng, Y. Lin, C. Su, and X. Zhang, "Omnidraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 42 319–42 348.

[43] N.-K. Le, T. D. Do, and L.-M. Nguyen, "SPECTRA: Faster large language model inference with optimized internal and external speculation," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 14 015–14 034.

[44] F. Huo, J. Tan, J. Liu, Z. Jiang, J. Li, J. Wang, X. Cai, and S. Sun, "Repspec: Structural re-parameterized draft model training for speculative decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[45] Y. Hu, J. Guo, X. Feng, and T. Zhao, "Adaspec: Selective knowledge distillation for efficient speculative

decoders," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 88 736–88 758.

[46] J. Fan, D. Cao, X. Luo, J. Fu, C. Liu, and X. Yang, "Flatter tokens are more valuable for speculative draft model training," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[47] Y.-Y. Qian, H.-C. Wu, Y. Fu, H. Zhang, and P. Zhao, "When drafts evolve: Speculative decoding meets online learning," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[48] J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen, and S. Mehrotra, "Draft & verify: Lossless large language model acceleration via self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 11 263–11 282.

[49] G. Monea, A. Joulin, and E. Grave, "PaSS: Parallel speculative sampling," in *Third Workshop on Efficient Natural Language and Speech Processing (ENLSP-III): Towards the Future of Large Language Models and Their Emerging Descendants*, 2023.

[50] H. M. Chen, W. Luk, K. F. C. Yiu, R. Li, K. Mishchenko, S. I. Venieris, and H. Fan, "Hardware-aware parallel prompt decoding for memory-efficient acceleration of LLM inference," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 2221–2238.

[51] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 28 935–28 948.

[52] C. Du, J. Jiang, X. Yuanchen, J. Wu, S. Yu, Y. Li, S. Li, K. Xu, L. Nie, Z. Tu, and Y. You, "GliDe with a CaPE: A low-hassle method to accelerate speculative decoding," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 11 704–11 720.

[53] Z. Ankner, R. Parthasarathy, A. Nrusimha, C. Rinard, J. Ragan-Kelley, and W. Brandon, "Hydra: Sequentially-dependent draft heads for medusa decoding," in *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024.

[54] Y. Cheng, A. Zhang, X. Zhang, C. Wang, and Y. Wang, "Recurrent drafter for fast speculative decoding in large language models," *ArXiv preprint*, vol. arXiv:2403.09919, 2024.

[55] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, S. Agarwal, A. Roman, A. Aly, B. Chen, and C.-J. Wu, "LayerSkip: Enabling early exit inference and self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 12 622–12 642.

[56] F. Liu, Y. Tang, Z. Liu, Y. Ni, D. Tang, K. Han, and Y. Wang, "Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 11 946–11 965.

[57] F. Gloeckle, B. Youbi Idrissi, B. Roziere, D. Lopez-Paz, and G. Synnaeve, "Better & faster large language models via multi-token prediction," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 15 706–15 734.

[58] W. Zhong and M. Bharadwaj, "S3d: A simple and cost-effective self-speculative decoding scheme for low-memory gpus," *ArXiv preprint*, vol. arXiv:2405.20314, 2024.

[59] J. Liu, Q. Wang, J. Wang, and X. Cai, "Speculative decoding via early-exiting for faster LLM inference with Thompson sampling control mechanism," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3027–3043.

[60] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-2: Faster inference of language models with dynamic draft trees," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 7421–7432.

[61] Z. Li, X. Yang, Z. Gao, J. Liu, G. Li, Z. Liu, D. Li, J. Peng, L. Tian, and E. Barsoum, "Amphista: Bi-directional multi-head decoding for accelerating LLM inference," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 8925–8938.

[62] B. Xiao, L. Gui, L. Su, and W. Chen, "Clover-2: Accurate inference for regressive lightweight speculative decoding," *ArXiv preprint*, vol. arXiv:2408.00264, 2024.

[63] K. Zhang, J. Zhao, and R. Chen, "KOALA: Enhancing speculative decoding for LLM via multi-layer draft heads with adversarial learning," in *28th International Conference on Computer Supported Cooperative Work in Design (CSCWD 2025)*, 2025, pp. 231–236.

[64] M. R. Metel, P. Lu, B. Chen, M. Rezagholizadeh, and I. Kobyzev, "Draft on the fly: Adaptive self-speculative decoding using cosine similarity," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 2267–2272.

[65] H. Xia, Y. Li, J. Zhang, C. Du, and W. Li, "SWIFT: On-the-fly self-speculative decoding for LLM inference acceleration," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[66] M. Zimmer, M. Gritta, G. Lampouras, H. B. Ammar, and J. Wang, "Mixture of attentions for speculative decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[67] S. Mehra, J. A. Garcia, and L. Mauch, "On multi-token prediction for efficient LLM inference," in *ICLR 2025 Workshop on Sparsity in LLMs (SLLM): Deep Dive into Mixture of Experts, Quantization, Hardware, and Inference*, 2025.

[68] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle-3: Scaling up inference acceleration of large language models via training-time test," in *Advances in Neural*

Information Processing Systems 38 (NeurIPS), 2025, pp. 136 737–136 756.

[69] J. Li, Y. Xu, H. Huang, X. Yin, D. Li, E. C. H. Ngai, and E. Barsoum, "Gumiho: A hybrid architecture to prioritize early tokens in speculative decoding," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 36 551–36 575.

[70] H. E. Zarch, L. Gao, C. Jiang, and M. Annavaram, "DEL: Context-aware dynamic exit layer for efficient self-speculative decoding," in *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

[71] L. Chen, R. Shan, H. Wang, L. Wang, Z. Liu, R. Luo, J. Wang, H. Alinejad-Rokny, and M. Yang, "CLaSp: In-context layer skip for self-speculative decoding," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 31 608–31 618.

[72] X. Liu, X. Xia, W. Zhao, M. Zhang, X. Yu, X. Su, S. Yang, S.-K. Ng, and T.-S. Chua, "L-mtp: Leap multi-token prediction beyond adjacent context for large language models," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 102 569–102 600.

[73] W. Zhong, M. Bharadwaj, Y. Wang, Y. Ji, and C. Lee, "Cross-attention speculative decoding," *ArXiv preprint*, vol. arXiv:2505.24544, 2025.

[74] Z. Wei, W.-L. Chen, X. Zhu, and Y. Meng, "AdaDecode: Accelerating LLM decoding with adaptive layer parallelism," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 65 981–65 996.

[75] M. Samragh, A. Kundu, D. Harrison, K. Nishu, D. Naik, M. Cho, and M. Farajtabar, "Your llm knows the future: Uncovering its multi-token prediction potential," *ArXiv preprint*, vol. arXiv:2507.11851, 2025.

[76] Z. Ning, J. Shao, R. Xu, X. Guo, J. Zhang, C. Zhang, and X. Li, "Cas-spec: Cascade adaptive self-speculative decoding for on-the-fly lossless inference acceleration of llms," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 65 145–65 167.

[77] J. Kim, S. Yu, G. Kim, and S. Kang, "FractalLLM: Lossless self-speculative decoding with layer embedded self-compression," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 23 666–23 673.

[78] N. Bhendawade, I. Belousova, Q. Fu, H. Mason, A. Lin, M. Rastegari, and M. Najibi, "Speculative streaming: Efficient and scalable speculative decoding with multi-stream attention," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 19 536–19 559.

[79] Y. Zhao, J. Tang, K. Zhu, Z. Ye, C.-C. Chang, C. Lin, J. Park, G. Xiao, M. S. Abdelfattah, M. Gao, B. Kasikci, S. Han, and I. Stoica, "Accelerating large-scale reasoning model inference with sparse self-speculative decoding," in *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026, pp. 251–265.

[80] J. Kirchenbauer, A. Hans, B. Bartoldson, M. Goldblum, A. Panda, and T. Goldstein, "Multi-token prediction via self-distillation," in *ICML 2026 Workshop on Resource-Adaptive Foundation Model Inference*, 2026.

[81] R. Goel, M. Gagrani, M. Lee, and C. Lott, "Efficient training-free multi-token prediction via embedding-space probing," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[82] A. Santilli, S. Severino, E. Postolache, V. Maiorca, M. Mancusi, R. Marin, and E. Rodola, "Accelerating transformer inference for translation via parallel decoding," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 12 336–12 355.

[83] Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He, "REST: Retrieval-based speculative decoding," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 1582–1595.

[84] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of LLM inference using lookahead decoding," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 14 060–14 079.

[85] W. Zhao, Y. Huang, X. Han, W. Xu, C. Xiao, X. Zhang, Y. Fang, K. Zhang, Z. Liu, and M. Sun, "Ouroboros: Generating longer drafts phrase by phrase for faster speculative decoding," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 13 378–13 393.

[86] S. Kou, L. Hu, Z. He, Z. Deng, and H. Zhang, "CLLMs: Consistency large language models," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 25 426–25 440.

[87] M. Li, X. Chen, A. Holtzman, B. Chen, J. Lin, W.-t. Yih, and X. V. Lin, "Nearest neighbor speculative decoding for llm generation and attribution," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 80 987–81 015.

[88] Y. Wang, X. Luo, F. Wei, Y. Liu, Q. Zhu, X. Zhang, Q. Yang, D. Xu, and W. Che, "Make some noise: Unlocking language model parallel inference capability through noisy training," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 12 914–12 926.

[89] X. Luo, Y. Wang, Q. Zhu, Z. Zhang, X. Zhang, Q. Yang, and D. Xu, "Turning trash into treasure: Accelerating inference of large language models with token recycling," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 6816–6831.

[90] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "Suffixdecoding: Extreme speculative decoding for emerging ai applications," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 126 326–126 354.

[91] Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen,

and J. Zhang, "SAM decoding: Speculative decoding via suffix automaton," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 12 187–12 204.

[92] M. Gritta, H. Xue, and G. Lampouras, "DReSD: Dense retrieval for speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 19 822–19 832.

[93] G. Chen, Q. Feng, J. Ni, X. Li, and M. Q. Shieh, "RAPID: Long-context inference with retrieval-augmented speculative decoding," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 8093–8107.

[94] S. Cho, S. Choi, T. Hwang, J. Seo, S. Jeong, H. Lee, H. Song, J. C. Park, and Y. Kwon, "Lossless acceleration of large language models with hierarchical drafting based on temporal locality in speculative decoding," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3895–3911.

[95] G. Quan, W. Feng, C. Hao, G. Jiang, Y. Zhang, and H. H. Wang, "RASD: Retrieval-augmented speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 6167–6177.

[96] J. Chen, Q. Li, Z. Li, B. Qi, G. Liu, H. Ai, H. Zhao, and P. Wang, "Faster in-context learning for LLMs via n-gram trie speculative decoding," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 18 040–18 051.

[97] Z. Ma, I. Gim, and L. Zhong, "Cacheback: Speculative decoding with nothing but cache," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 31 079–31 084.

[98] L. Hu, S. Kou, Y. Fu, S. Rajbhandari, T. Rosing, Y. He, Z. Deng, and H. Zhang, "Fast and accurate causal parallel decoding using jacobi forcing," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[99] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024, pp. 932–949.

[100] Z. Sun, A. T. Suresh, J. H. Ro, A. Beirami, H. Jain, and F. Yu, "Spectr: Fast speculative decoding via optimal transport," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 30 222–30 242.

[101] S. Yang, S. Huang, X. Dai, and J. Chen, "Multi-candidate speculative decoding," in *Natural Language Processing and Chinese Computing – 14th National CCF Conference, NLPCC 2025, Proceedings, Part I*, 2025, pp. 335–348.

[102] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable and robust speculative decoding," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 129 531–129 563.

[103] Z. Hu and H. Huang, "Accelerated speculative sampling based on tree Monte Carlo," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 19 216–19 251.

[104] Z. Sun, U. Mendlovic, Y. Leviathan, A. Aharoni, J. H. Ro, A. Beirami, and A. T. Suresh, "Block verification accelerates speculative decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[105] C.-H. Lin, S. Tuli, J. Smith, Y.-C. Hsu, Y. Shen, and H. Jin, "SLiM: Speculative decoding with hypothesis reduction," in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024, pp. 1005–1017.

[106] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat, and R. Schwartz, "Dynamic speculation lookahead accelerates speculative decoding of large language models," in *Proceedings of The 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, 2024, pp. 456–467.

[107] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "OPT-tree: Speculative decoding with adaptive draft tree structure," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 188–199, 2025.

[108] Z. Gong, J. Liu, Z. Wang, P. Wu, J. Wang, X. Cai, D. Zhao, and R. Yan, "Graph-structured speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 11 404–11 415.

[109] O. Brown, Z. Wang, A. Do, N. Mathew, and C. Yu, "Dynamic depth decoding: Faster speculative decoding for llms," *ArXiv preprint*, vol. arXiv:2409.00142, 2024.

[110] A. J. Khisti, M. Ebrahimi, H. Dbouk, A. Behboodi, R. Memisevic, and C. Louizos, "Multi-draft speculative sampling: Canonical decomposition and theoretical limits," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[111] M. Yin, M. Chen, K. Huang, and M. Wang, "A theoretical perspective for speculative decoding algorithm," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 128 082–128 117.

[112] R. Sun, T. Zhou, X. Chen, and L. Sun, "SpecHub: Provable acceleration to multi-draft speculative decoding," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 20 620–20 641.

[113] Z. Zhang, J. Xu, T. Liang, X. Chen, Z. He, R. Wang, and Z. Tu, "Draft model knows when to stop: Self-verification speculative decoding for long-form generation," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 16 685–16 697.

[114] S. Zhang, H. Wang, D. Ma, Z. Zhu, L. Chen, K. Lan, and K. Yu, "Adaeagle: Optimizing speculative decoding via explicit modeling of adaptive draft structures," *ArXiv preprint*, vol. arXiv:2412.18910, 2024.

[115] G. Bachmann, S. Anagnostidis, A. Pumarola, M. Georgopoulos, A. Sanakoyeu, Y. Du, E. Schönfeld, A. Tha-

bet, and J. K. Kohler, "Judge decoding: Faster speculative sampling requires going beyond model alignment," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[116] Z. Wu, Z. Zhou, A. Verma, A. Prakash, D. Rus, and B. K. H. Low, "TETRIS: Optimal draft token selection for batch speculative decoding," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 33 329–33 345.

[117] Z. Hu, T. Zheng, V. Viswanathan, Z. Chen, R. A. Rossi, Y. Wu, D. Manocha, and H. Huang, "Towards optimal multi-draft speculative decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[118] M. Holsman, Y. Huang, and B. Dhingra, "Fuzzy speculative decoding for a tunable accuracy-runtime tradeoff," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 26 257–26 273.

[119] S. Syu and H.-y. Lee, "Hierarchical speculative decoding with dynamic window," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 8275–8288.

[120] S. Kobus and D. Gunduz, "Speculative sampling via exponential races," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 18 189–18 204.

[121] J. Wang, Z. Tian, J. Li, Q. Xia, X. Duan, Z. Wang, B. Huai, and M. Zhang, "Alignment-augmented speculative decoding with alignment sampling and conditional verification," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 6751–6763.

[122] Y. Weng, Q. Hu, X. Chen, L. Liu, D. Mei, H. Qiu, J. Tian, and Z. Shi, "Traversal verification for speculative tree decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 77 899–77 924.

[123] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "Stree: Speculative tree decoding for hybrid state space models," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 115 588–115 608.

[124] Y. Hou, F. Zhang, C. Du, X. Zhang, J. Pan, T. Pang, C. Du, V. Tan, and Z. Yang, "BanditSpec: Adaptive speculative decoding via bandit algorithms," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 24 045–24 079.

[125] S. Liu, Y. Ye, Q. Zhu, Z. Cao, and Y. He, "HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, pp. 12 930–12 947.

[126] J. Rowan, T. B. Phan, and A. Khisti, "List-level distribution coupling with applications to speculative decoding and lossy compression," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 5576–5619.

[127] Y. Shen, J. Shen, Q. Kong, T. Liu, Y. Lu, and C. Wang, "Specbranch: Speculative decoding via hybrid drafting and rollback-aware branch parallelism," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[128] J. Liu, B. Park, and X. Shen, "A drop-in solution for on-the-fly adaptation of speculative decoding in large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 9778–9794.

[129] H. Zheng and X. Wang, "Faster speculative decoding via effective draft decoder with pruned candidate tree," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 9856–9868.

[130] S. Hu, J. Li, Z. Lu, and P. Zhou, "Bridging draft policy misalignment: Group tree optimization for speculative decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[131] R. Wang, H. Li, Y. Ma, X. Zheng, F. Chao, X. Xiao, and R. Ji, "polybasic speculative decoding through a theoretical perspective," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 63 387–63 398.

[132] H. Liu, J. Huang, Z. Jia, Y. Park, and Y.-X. Wang, "Not-a-bandit: Provably no-regret drafter selection in speculative decoding for LLMs," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[133] R. K. Thomas and A. Pal, "Global resolution: Optimal multi-draft speculative sampling via convex optimization," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[134] J. Li, Y. Xu, G. Li, S. Yang, J. Xu, X. Yin, D. Li, E. C. H. Ngai, and E. Barsoum, "Training-free loosely speculative decoding: Accepting semantically correct drafts beyond exact match," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[135] Y. Zhou, F. Huang, H. Li, F. Wu, T. Wang, J. Zhang, J. Lin, and Z.-Q. Cheng, "Overcoming joint intractability with lossless hierarchical speculative decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[136] J. Zhang, Z. Yu, L. Wang, N. Yang, E. J. Yu, Z. Li, Y. Song, D. Zhu, X. Zhang, F. Wei, and S. Li, "Learning to draft: Adaptive speculative decoding with reinforcement learning," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[137] T. Kumar, T. Dao, and A. May, "Speculative speculative decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[138] Y. Hao and L. Mou, "Cactus: Accelerating auto-regressive decoding with constrained acceptance speculative sampling," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[139] W. Shi, Q. Xu, F. Deng, Y. Wu, J. Liu, Y. Xu, H. Chen, J. Zhu, J. Xu, X. Huang, J. Yang, and X. Zhou, "Specblock: Block-iterative speculative decod-

ing with dynamic tree drafting," *ArXiv preprint*, vol. arXiv:2605.07243, 2026.

[140] L. Hu, Z. Feng, Y. Wu, H. Yuan, Y. Zhao, Y.-Y. Qian, B. Wang, P. Zhao, D. Jiang, Y. Zhu, T. Rosing, and H. Zhang, "Jetspec: Breaking the scaling ceiling of speculative decoding with parallel tree drafting," *ArXiv preprint*, vol. arXiv:2606.18394, 2026.

[141] K. Huang, X. Guo, and M. Wang, "Specdec++: Boosting speculative decoding via adaptive candidate lengths," in *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

[142] R. Pan, Z. Chen, H. Liu, A. Krishnamurthy, and R. Netravali, "Fail fast, win big: Rethinking the drafting strategy in speculative decoding via diffusion LLMs," *ArXiv preprint*, vol. arXiv:2512.20573, 2025.

[143] N. S. Nakshatri, S. Roy, R. Das, S. Chaidaroon, L. Boytsov, and R. Gangadharaiah, "Constrained decoding with speculative lookaheads," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 4681–4700.

[144] Y. Hu, T. Xia, Z. Liu, R. Raman, X. Liu, B. Bao, E. Sather, V. Thangarasa, and S. Q. Zhang, "Dream: Drafting with refined target features and entropy-adaptive cross-attention fusion for multimodal speculative decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 167 592–167 612.

[145] M. Ganesan, S. Segal, A. Aggarwal, N. Sinnadurai, S. Lie, and V. Thangarasa, "MASSV: Multimodal adaptation and self-data distillation for speculative decoding of vision-language models," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 12 265–12 276.

[146] S. Wang, R. Yu, Z. Yuan, C. Yu, F. Gao, Y. Wang, and D. F. Wong, "Spec-VLA: Speculative decoding for vision-language-action models with relaxed acceptance," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 26 928–26 940.

[147] Y. Ji, J. Zhang, H. Xia, J. Chen, L. Shou, G. Chen, and H. Li, "SpecVLM: Enhancing speculative decoding of video LLMs via verifier-guided token pruning," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 7205–7219.

[148] J. Kang, H. Shu, W. Li, Y. Zhai, and X. Chen, "Vispec: Accelerating vision-language models with vision-aware speculative decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 115 511–115 532.

[149] B. Liu, A. Wang, Z. Min, L. Yao, H. Zhang, Y. Liu, X. Han, P. Li, A. Zeng, and J. Su, "Spec-rl: Accelerating on-policy reinforcement learning with speculative rollouts," *ArXiv preprint*, vol. arXiv:2509.23232, 2025.

[150] Y. Zhang, N. Lv, T. Wang, and J. Dang, "FastGRPO: Accelerating policy optimization via concurrency-aware speculative decoding and online draft learning," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[151] Q. Chen, Z. Liu, P. Sun, S. Li, G. Wang, Z. Liu, Y. Wen, S. Feng, and T. Zhang, "Respec: Towards optimizing speculative decoding in reinforcement learning systems," in *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026, pp. 367–379.

[152] Z. Shao, V. Srivatsa, S. Srivastava, Q. Wu, A. Ariyak, X. Wu, A. Patel, J. Wang, P. Liang, T. Dao, C. Zhang, Y. Zhang, B. Athiwaratkun, C. Xu, and J. Wang, "Beat the long tail: Distribution-aware speculative decoding for RL training," in *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026, pp. 1626–1638.

[153] R. Cheng, K. Zhou, X. Wei, S. Liu, M. Han, M. Ai, Y. Zhou, B. Zhong, W. Xiao, R. Chen, and H. Chen, "Fast llm post-training via decoupled and fastest-of-n speculation," *ArXiv preprint*, vol. arXiv:2511.16193, 2025.

[154] Q. Hu, S. Yang, J. Guo, X. Yao, Y. Lin, Y. Gu, H. Cai, C. Gan, A. Klimovic, and S. Han, "Taming the long-tail: Efficient reasoning RL training with adaptive drafter," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2026, pp. 1933–1948.

[155] S. Wang, H. Yang, J. Zhu, X. Wang, Y. Xu, and D. Qian, "Rlhfspec: Breaking the efficiency bottleneck in rlhf training via adaptive drafting," *ArXiv preprint*, vol. arXiv:2512.04752, 2025.

[156] C.-C. Chang, S. Zhu, Z. Zeng, H. Lin, X. Liu, J. You, M. S. Abdelfattah, Z. Jiang, and X. Qian, "SRT: Accelerating reinforcement learning via speculative rollout with tree-structured cache," in *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025.

[157] H. Yuan, K. Lu, F. Huang, Z. Yuan, and C. Zhou, "Speculative contrastive decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 56–64.

[158] B. Li, Y. Wu, X. Luo, and R. Zhang, "Reward-shifted speculative sampling is an efficient test-time weak-to-strong aligner," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 11 468–11 478.

[159] X. Wang, S. Zhu, and X. Cheng, "Speculative safety-aware decoding," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 12 827–12 841.

[160] Z. Hu and H. Huang, "Inevitable trade-off between watermark strength and speculative sampling efficiency for language models," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 55 370–55 402.

[161] W. He, X. Li, L. Shen, W. J. Su, and Q. Long, "Improving the trade-off between watermark strength and speculative sampling efficiency for language models," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[162] H. Zhang, Z. Liu, Y. Zhao, J. Zheng, C. Zhuang, J. Gu, and G. Chen, "Fast chain-of-thought: A glance of future from parallel decoding leads to answers faster," *ArXiv preprint*, vol. arXiv:2311.08263, 2023.

[163] Z. Wang, J. Wu, Y. Lai, C. Zhang, and D. Zhou, "SEED: Accelerating reasoning tree construction via scheduled speculative decoding," in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025, pp. 4920–4937.

[164] Z. Wang, Z. Wang, L. T. Le, H. S. Zheng, S. Mishra, V. Perot, Y. Zhang, A. Mattapalli, A. Taly, J. Shang, C.-Y. Lee, and T. Pfister, "Speculative RAG: Enhancing retrieval augmented generation through drafting," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[165] B. Liao, Y. Xu, H. Dong, J. Li, C. Monz, S. Savarese, D. Sahoo, and C. Xiong, "Reward-guided speculative decoding for efficient LLM reasoning," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 37 555–37 572.

[166] Y. Li, J. Shi, S. Feng, P. Yuan, X. Wang, Y. Zhang, J. Zhang, C. Tan, B. Pan, Y. Hu, and K. Li, "Speculative decoding for multi-sample inference," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 12 523–12 533.

[167] R. Pan, Y. Dai, Z. Zhang, G. Oliaro, Z. Jia, and R. Netravali, "Specreason: Fast and accurate inference-time compute via speculative reasoning," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 12 730–12 749.

[168] W. Yang, X. Yue, V. Chaudhary, and X. Han, "Speculative thinking: Enhancing small-model reasoning with large model guidance at inference time," in *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

[169] J. Wang, J. Li, J. Hou, Y. Bowen, L. Wu, and M. Zhang, "Efficient reasoning for LLMs through speculative chain-of-thought," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 1541–1553.

[170] Z. Wang, J. Wang, J. Pan, X. Xia, H. Zhen, M. Yuan, J. Hao, and F. Wu, "Accelerating large language model reasoning via speculative search," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 64 773–64 805.

[171] T. Fu, Y. Ge, Y. You, E. Liu, Z. Yuan, G. Dai, S. Yan, H. Yang, and Y. Wang, "R2r: Efficiently navigating divergent reasoning paths with small-large model token routing," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 124 108–124 145.

[172] Y. Fu, R. Ge, Z. Shao, Z. Deng, and H. Zhang, "Scaling speculative decoding with lookahead reasoning," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 172 454–172 486.

[173] W. Song, S. Dingliwal, S. M. Jayanthi, B. Ganesh, J. Shin, A. Galstyan, and S. B. Bodapati, "Accelerated test-time scaling with model-free speculative sampling," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 30 623–30 636.

[174] R. Yang, H. Bai, S. Liu, G. Yu, R. Fan, Y. Dang, J. Zhang, K. Liu, J. Zhu, and P. Chen, "Specexit: Accelerating large reasoning model via speculative exit," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[175] J. Shi, Y. Zhu, Z. Shi, D. Zhao, Q. Li, and Y. Jiang, "SpecCoT: Accelerating chain-of-thought reasoning through speculative exploration," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 24 405–24 415.

[176] X. Dong, S. Wang, D. Lin, B. Chen, and A. E. Hassan, "Beyond tokens: Semantic-aware speculative decoding for efficient inference by probing internal states," *ArXiv preprint*, vol. arXiv:2602.03708, 2026.

[177] K. Purohit, R. Narayanam, and S. Pal, "From tokens to steps: Verification-aware speculative decoding for efficient multi-step reasoning," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 17 457–17 471.

[178] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, and Y. Wang, "Skeleton-of-thought: Prompting LLMs for efficient parallel generation," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[179] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI)*, 2024, pp. 17 682–17 690.

[180] S. Jin, X. Liu, Y. Wu, H. Zheng, Q. Zhang, A. Prakash, M. Lentz, D. Zhuo, F. Qian, and Z. M. Mao, "Plato: Plan to efficiently decode for large language model inference," in *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

[181] T. Jin, E. Y. Cheng, Z. Ankner, N. Saunshi, B. M. Elias, A. Yazdanbakhsh, J. Ragan-Kelley, S. Subramanian, and M. Carbin, "Learning to keep a promise: Scaling language model decoding parallelism with learned asynchronous decoding," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 27 941–27 956.

[182] Y. Yu, W. Wang, R. Chen, and J. Pei, "Accelerate parallelizable reasoning via parallel decoding within one sequence," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 9007–9014.

[183] J. Pan, X. Li, L. Lian, C. Snell, Y. Zhou, A. Yala, T. Darrell, K. Keutzer, and A. Suhr, "Learning adaptive parallel reasoning with language models," in *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

[184] X. Yang, Y. An, H. Liu, T. Chen, and B. Chen, "Multiverse: Your language models secretly decide how to parallelize and merge generation," in *Advances in*

Neural Information Processing Systems 38 (NeurIPS), 2025, pp. 100 773–100 804.

[185] E. Biju, S. Talaei, Z. Huang, M. Pourreza, A. Mirhoseini, and A. Saberi, "Sprint: Enabling interleaved planning and parallelized execution in reasoning models," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 96 393–96 433.

[186] H. Wu, Z. Teng, and K. Tu, "Parallel continuous chain-of-thought with jacobi iteration," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 914–926.

[187] Q. Ai and H. Jiang, "Parallel thinking, sequential answering: Bridging nar and ar for efficient reasoning," *ArXiv preprint*, vol. arXiv:2509.20744, 2025.

[188] B. Wu, M. Chen, X. Luo, S. Yan, Q. Yu, F. Xia, T. Zhang, H. Zhan, Z. Zhong, X. Zhou, S. Qiao, and X. Bin, "Parallel loop transformer for efficient test-time computation scaling," *ArXiv preprint*, vol. arXiv:2510.24824, 2025.

[189] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More agents is all you need," *Transactions on Machine Learning Research*, 2024.

[190] H. Sun, M. Haider, R. Zhang, H. Yang, J. Qiu, M. Yin, M. Wang, P. L. Bartlett, and A. Zanette, "Fast best-of-n decoding via speculative rejection," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 32 630–32 652.

[191] Y. Ding, W. Jiang, S. Liu, Y. Jing, J. Guo, Y. Wang, J. Zhang, Z. Wang, Z. Liu, B. Du, X. Liu, and D. Tao, "Dynamic parallel tree search for efficient LLM reasoning," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 11 233–11 252.

[192] C.-J. Hsu, D. Buffelli, J. McGowan, F.-T. Liao, Y.-C. Chen, S. Vakili, and D. shan Shiu, "Group think: Multiple concurrent reasoning agents collaborating at token level granularity," *ArXiv preprint*, vol. arXiv:2505.11107, 2025.

[193] Z. Xu, Z. Qiu, G. Huang, K. Li, S. Li, C. Zhang, K. Li, Q. Yi, Y. Jiang, B. Zhou, F. Lian, and Z. Kang, "Adaptive termination for multi-round parallel reasoning: An universal semantic entropy-guided framework," *ArXiv preprint*, vol. arXiv:2507.06829, 2025.

[194] Z. Wang, B. Niu, Z. Li, L. Meng, J. Liu, Z. Zheng, T. Xu, H. Wu, H. Wang, and E. Chen, "A2r: An asymmetric two-stage reasoning framework for parallel reasoning," *ArXiv preprint*, vol. arXiv:2509.22044, 2025.

[195] T. Zheng, H. Zhang, W. Yu, X. Wang, H. Xing, R. Dai, R. Liu, H. Bao, C. Huang, H. Huang, and D. Yu, "Parallel-r1: Towards parallel thinking via reinforcement learning," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[196] J. Xiong, Q. Chen, F. Ye, Z. Wan, C. Zheng, C. Zhao, H. Shen, A. H. Li, C. Tao, H. Tan, H. Bai, L. Shang, L. Kong, and N. Wong, "ATTS: Asynchronous test-time scaling via conformal prediction," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[197] R. You, Y. Li, M. Liu, W. Wang, L. Nie, and W. Li, "Parallel test-time scaling for latent reasoning models," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, pp. 44 703–44 717.

[198] S. Tu, Y. Li, Y. Bai, L. Hou, and J. Li, "DeepPrune: Parallel scaling without inter-trace redundancy," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 13 389–13 403.

[199] Z. Xu, X. Lou, G. Wang, Y.-N. Chuang, F. Luo, G. Zheng, A. S. Szalay, Z. Liu, and V. Braverman, "DTS: Enhancing large reasoning models via decoding tree sketching," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[200] T. Zheng, C. Huang, R. Dai, Y. He, R. Liu, X. Ni, H. Bao, K. Wang, H. Zhu, J. Huang, F. Huang, and H. Huang, "Parallel-probe: Towards efficient parallel thinking via 2d probing," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[201] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen LLM applications via multi-agent conversations," in *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024.

[202] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[203] E. Zhang, E. Zhu, G. Bansal, A. Fourney, H. Mozannar, and J. Gerrits, "Optimizing sequential multi-step tasks with parallel LLM agents," in *ICML 2025 Workshop on Multi-Agent Systems in the Era of Foundation Models: Opportunities, Challenges and Futures*, 2025.

[204] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An LLM compiler for parallel function calling," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 24 370–24 391.

[205] S. Singh, A. Karatzas, M. Fore, I. Anagnostopoulos, and D. Stamoulis, "An llm-tool compiler for fused parallel function calling," *ArXiv preprint*, vol. arXiv:2405.17438, 2024.

[206] W. Hua, M. Wan, J. S. S. S. Vadrevu, R. Nadel, Y. Zhang, and C. Wang, "Interactive speculative planning: Enhance agent efficiency through co-design of system and user interface," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[207] A. A. Ginart, N. Kodali, J. Lee, C. Xiong, S. Savarese, and J. Emmons, "Asynchronous tool usage for real-time agents," *ArXiv preprint*, vol. arXiv:2410.21620, 2024.

[208] I. Gim, S. seob Lee, and L. Zhong, "Asynchronous llm function calling," *ArXiv preprint*, vol.

arXiv:2412.07017, 2024.

[209] Y. Guan, Q. Lan, F. Sun, D. Ding, D. Acharya, C. Wang, W. Y. Wang, and W. Hua, "Dynamic speculative agent planning," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[210] N. Ye, A. Ahuja, G. Liargkovas, Y. Lu, K. Kaffes, and T. Peng, "Speculative actions: A lossless framework for faster agentic systems," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[211] Z. Huang, W. Zeng, T. Fu, T. Liu, Y. Sun, K. Hong, X. Yang, C. Liu, Y. Li, Q. Zhang, G. Dai, Z. Zhu, and Y. Wang, "Reducing latency of LLM search agent via speculation-based algorithm-system co-design," *ArXiv preprint*, vol. arXiv:2511.20048, 2025.

[212] D. Nichols, P. Singhania, C. Jekel, A. Bhatele, and H. Menon, "Optimizing agentic language model inference via speculative tool calls," *ArXiv preprint*, vol. arXiv:2512.15834, 2025.

[213] X. Lin, J. H. Liew, S. Savarese, and J. Li, "W&D: Scaling parallel tool calling for efficient deep research agents," in *ICLR 2026 Workshop on Agents in the Wild (AIWILD)*, 2026.

[214] X. Shi, J. Wan, L. Dong, W. Jiang, Y. Liu, and Z. Huang, "Realtimetool: Parallel decoding for real-time LLM function calling," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[215] S. Zhong, B. Lu, Q. Chen, C. Liu, F. Yang, and M. Li, "Dualspec: Accelerating deep research agents via dual-process action speculation," *ArXiv preprint*, vol. arXiv:2603.07416, 2026.

[216] Y. Sui, H. Zhao, R. Ma, Z. He, H. Wang, J. Li, K. Xu, K. Chen, and Y. Yang, "Parallelizing tool execution and llm generation for low-latency agent serving," *ArXiv preprint*, vol. arXiv:2603.18897, 2026.

[217] H. Bai, W. Lv, H. Zheng, Y. Lu, and J. Shu, "SPORK: Self-speculative forking to accelerate agentic LLM inference," *ArXiv preprint*, vol. arXiv:2607.03333, 2026.

[218] Y. Li, Q. Ye, P. K. Choubey, J. Zhang, and C.-S. Wu, "Speculate with memory: Lossless acceleration for LLM agents," *ArXiv preprint*, vol. arXiv:2607.12236, 2026.

[219] J. Ji, Y. Liu, L. An, R. Jain, G. Polatkan, S. Zhu, and S. Chang, "Speculate while you reason: Teaching agents to predict their next tool call via joint agent-speculator RL," *ArXiv preprint*, vol. arXiv:2607.25816, 2026.

[220] Z. Zhang, M. Xu, F. Sun, and Z. Zhang, "OoO-Spec: Out-of-order semantic speculation for fast tool calling," *ArXiv preprint*, vol. arXiv:2608.00814, 2026.

[221] H. M. Chen, J. Guo, W. Luk, and H. Fan, "AOSpec: Action and observation co-speculation for low-latency agent serving," *ArXiv preprint*, vol. arXiv:2608.00881, 2026.

[222] Z. Liu, S. Kundu, and P. A. Beerel, "Speculative macro commit for faster tool-using agents," in *IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, 2026, p. to appear.

[223] Z. Xu, Q. Zhang, G. Wan, S. Zhu, and C. Zhang, "LEAP: Learning efficient action proposals for LLM agents," *ArXiv preprint*, vol. arXiv:2610.02670, 2026.

[224] J. Gu, J. Bradbury, C. Xiong, V. O. K. Li, and R. Socher, "Non-autoregressive neural machine translation," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.

[225] J. Lee, E. Mansimov, and K. Cho, "Deterministic non-autoregressive neural sequence modeling by iterative refinement," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 1173–1182.

[226] L. Kaiser, S. Bengio, A. Roy, A. Vaswani, N. Parmar, J. Uszkoreit, and N. Shazeer, "Fast decoding in sequence models using discrete latent variables," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 2390–2399.

[227] M. Ghazvininejad, O. Levy, Y. Liu, and L. Zettlemoyer, "Mask-predict: Parallel decoding of conditional masked language models," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 6112–6121.

[228] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, "Structured denoising diffusion models in discrete state-spaces," in *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021, pp. 17 981–17 993.

[229] X. L. Li, J. Thickstun, I. Gulrajani, P. Liang, and T. B. Hashimoto, "Diffusion-LM improves controllable text generation," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 4328–4343.

[230] A. Lou, C. Meng, and S. Ermon, "Discrete diffusion modeling by estimating the ratios of the data distribution," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 32 819–32 848.

[231] J. Shi, K. Han, Z. Wang, A. Doucet, and M. K. Titsias, "Simplified and generalized masked diffusion for discrete data," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 103 131–103 167.

[232] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov, "Simple and effective masked diffusion language models," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 130 136–130 184.

[233] J. Ou, S. Nie, K. Xue, F. Zhu, J. Sun, Z. Li, and C. Li, "Your absorbing discrete diffusion secretly models the conditional distributions of clean data," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[234] J. Deschenaux and C. Gulcehre, "Beyond autoregression: Fast llms via self-distillation through time," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[235] S. S. Sahoo, J. Deschenaux, A. Gokaslan, G. Wang, J. T. Chiu, and V. Kuleshov, "The diffusion duality," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 52 584–52 619.

[236] S. Gong, S. Agarwal, Y. Zhang, J. Ye, L. Zheng, M. Li, C. An, P. Zhao, W. Bi, H. Peng, J. Han, and L. Kong, "Scaling diffusion language models via adaptation from autoregressive models," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[237] M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov, "Block diffusion: Interpolating between autoregressive and diffusion language models," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[238] Y. Liu, Y. Cao, H. Li, G. Luo, Z. Chen, W. Wang, X. Liang, B. Qi, L. Wu, C. Tian, Y. Zhang, Y. Li, T. Lu, Y. Qiao, J. Dai, and W. Wang, "Sequential diffusion language models," *ArXiv preprint*, vol. arXiv:2509.24007, 2025.

[239] C. Wu, H. Zhang, S. Xue, S. Diao, Y. Fu, Z. Liu, P. Molchanov, P. Luo, S. Han, and E. Xie, "Fast-dLLM v2: Efficient block-diffusion LLM," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[240] Radical Numerics Inc., "Training diffusion language models at scale using autoregressive models," Radical Numerics Inc., Tech. Rep., 2025, https://github.com/RadicalNumerics/RND1.

[241] M. Arriola, Y. Schiff, H. Phung, A. Gokaslan, and V. Kuleshov, "Encoder-decoder diffusion language models for efficient training and inference," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 15 612–15 644.

[242] S. Cheng, Y. Bian, D. Liu, Y. Jiang, Y. Liu, L. Zhang, Q. Yao, Z. Tian, W. Wang, Q. Guo, K. Chen, B. Qi, and B. Zhou, "SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 22 058–22 075.

[243] Y. Tian, Y. Liang, S. Zhang, Y. Shu, G. Yang, W. He, S. Fang, T. Guo, K. Han, C. Xu, H. Chen, X. Chen, and Y. Wang, "From next-token to next-block: A principled adaptation path for diffusion llms," *ArXiv preprint*, vol. arXiv:2512.06776, 2025.

[244] Y. Fu, L. Whalen, Z. Ye, X. Dong, S. Diao, J. Liu, C. Wu, H. Zhang, E. Xie, S. Han, M. Khadkevich, J. Kautz, Y. C. Lin, and P. Molchanov, "Efficient-dlm: From autoregressive to diffusion language models, and beyond in speed," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[245] T. Bie, M. Cao, K. Chen, L. Du, M. Gong, Z. Gong, Y. Gu, J. Hu, Z. Huang, Z. Lan, C. Li, C. Li, J. Li, Z. Li, H. Liu, L. Liu, G. Lu, X. Lu, Y. Ma, J. Tan, L. Wei, J.-R. Wen, Y. Xing, X. Zhang, J. Zhao, D. Zheng, J. Zhou, J. Zhou, Z. Zhou, L. Zhu, and Y. Zhuang, "Llada2.0: Scaling up diffusion language models to 100b," *ArXiv preprint*, vol. arXiv:2512.15745, 2025.

[246] L. Ma, Y. Cui, K. Han, and Y. Wang, "Diffusion in diffusion: Reclaiming global coherence in semi-autoregressive diffusion," *ArXiv preprint*, vol. arXiv:2601.13599, 2026.

[247] Y. Jin, J. Xu, Y. Liu, C. Xu, Y. Tu, J. Li, D. Tu, X. Yan, K. Yu, P. Liu, and Z. Deng, "Multi-block diffusion language models," *ArXiv preprint*, vol. arXiv:2606.29215, 2026.

[248] H. Ben-Hamu, I. Gat, D. Severo, N. S. Nolte, and B. Karrer, "Accelerated sampling from masked diffusion models via entropy bounded unmasking," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 55 981–56 007.

[249] C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie, "Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[250] Q. Wei, Y. Zhang, Z. Liu, P. Zeng, Y. Wang, B. Qi, D. Liu, and L. Zhang, "Accelerating diffusion large language models with slowfast sampling: The three golden principles," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[251] F. Hong, G. Yu, Y. Ye, H. Huang, H. Zheng, Y. Zhang, Y. Wang, and J. Yao, "Wide-in, narrow-out: Revokable decoding for efficient and effective DLLMs," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[252] P. Li, Y. Zhou, D. Muhtar, L. Yin, S. Yan, L. Shen, S. Vosoughi, and S. Liu, "Diffusion language models know the answer before decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[253] D. Gwak, M. Jung, J. Park, M. Park, C. Park, J. Hyung, and J. Choo, "Reward-weighted sampling: Enhancing non-autoregressive characteristics in masked diffusion LLMs," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 34 574–34 594.

[254] F. Fu, T. Guo, and Z. Liu, "Learnable sampler distillation for discrete diffusion models," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 70 161–70 190.

[255] P. Bansal and S. Sanghavi, "Enabling approximate joint sampling in diffusion lms," *ArXiv preprint*, vol. arXiv:2509.22738, 2025.

[256] W. Bao, Z. Chen, D. Xu, and Y. Shang, "Learning to parallel: Accelerating diffusion large language models via learnable parallel decoding," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[257] Z. Chen, G. Fang, X. Ma, R. Yu, and X. Wang, "dparallel: Learnable parallel decoding for dLLMs," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[258] S. Wu and J. Zhang, "Free draft-and-verification: To-

ward lossless parallel decoding for diffusion large language models," in *The First Workshop on Efficient Reasoning, NeurIPS 2025*, 2025.

[259] F. Kong, J. Zhang, Y. Liu, Z. Wu, Y. Tian, V. W., and G. Zhou, "Accelerating diffusion llm inference via local determinism propagation," *ArXiv preprint*, vol. arXiv:2510.07081, 2025.

[260] Y. Dong, Z. Ma, X. Jiang, Z. Fan, J. Qian, Y. Li, J. Xiao, Z. Jin, and G. Li, "Saber: Efficient sampling with adaptive acceleration and backtracking enhanced remasking for diffusion language model in code generation," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, pp. 3623–3642.

[261] C. Xu, Y. Jin, J. Li, Y. Tu, G. Long, D. Tu, M. Song, H. Si, T. Hou, J. Yan, and Z. Deng, "Lopa: Scaling dllm inference via lookahead parallel decoding," *ArXiv preprint*, vol. arXiv:2512.16229, 2025.

[262] A. Mohamed, Y. Zhang, M. Vazirgiannis, and G. Shang, "Fast-decoding diffusion language models via progress-aware confidence schedules," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 35 793–35 807.

[263] J. Shen, G. Sarkar, Y. Ro, S. N. Sridhar, Z. Wang, A. Akella, and S. Kundu, "Improving the throughput of diffusion-based large language models via a training-free confidence-aware calibration," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 9826–9837.

[264] M. Jazbec, T. X. Olausson, L. Béthune, P. Ablin, M. Kirchhof, J. Monteiro, V. G. T. da Costa, J. Ramapuram, and M. Cuturi, "Learning unmasking policies for diffusion language models," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[265] Y. Shen, T. Feng, J. Han, W. Wang, T. Chen, C. Shen, J. Leskovec, and S. Ermon, "Improving diffusion language model decoding through joint search in generation order and token space," *ArXiv preprint*, vol. arXiv:2601.20339, 2026.

[266] Y.-Y. Qian, J. Su, L. Hu, P. Zhang, Z. Deng, P. Zhao, and H. Zhang, "d3LLM: Ultra-fast diffusion LLM using pseudo-trajectory distillation," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[267] X. Wang, M. Zhang, S. Cui, Z. Chen, B. Jiang, K. Kuang, and M. Lin, "Reversible diffusion decoding for diffusion language models," *ArXiv preprint*, vol. arXiv:2602.00150, 2026.

[268] Y. Ye, F. Hong, H. Zheng, X. Chen, Z. Chen, Y. Wang, and J. Yao, "Rejection mixing: Fast semantic propagation of mask tokens for efficient DLLM inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026, pp. 17 619–17 629.

[269] K. Yang, J. Teoh, K. Yang, Y. Zhang, and A. Lamb, "Improving sampling for masked diffusion models via information gain," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[270] X. Zhou, Y. Hu, and J. Huang, "DOS: Dependency-oriented sampler for masked diffusion language models," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 17 404–17 419.

[271] C. Cai and G. Li, "Confidence-based decoding is provably efficient for diffusion language models," *ArXiv preprint*, vol. arXiv:2603.22248, 2026.

[272] H. Sun, Z. Chen, X. Yang, Y. Tian, and B. Chen, "Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding," in *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024.

[273] H. Qian, S. K. Gonugondla, S. Ha, M. Shang, S. K. Gouda, R. Nallapati, S. Sengupta, X. Ma, and A. Deoras, "BASS: Batched attention-optimized speculative sampling," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 8214–8224.

[274] N. Timor, J. Mamou, D. Korat, M. Berchansky, O. Pereg, M. Wasserblat, T. Galanti, M. Gordon-Kiwkowitz, and D. Harel, "Distributed speculative inference (DSI): Speculation parallelism for provably faster lossless language model inference," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[275] Y. Ni, C. Liu, Y. Tang, K. Han, and Y. Wang, "EMS-SD: Efficient multi-sample speculative decoding for accelerating large language models," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 9307–9320.

[276] R. Svirschevski, A. May, Z. Chen, B. Chen, Z. Jia, and M. Ryabinin, "Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 16 342–16 368.

[277] D. Wagner, S. Lee, I. Baumann, P. Seeberger, K. Riedhammer, and T. Bocklet, "Optimized speculative sampling for GPU hardware accelerators," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 6442–6458.

[278] R. Sadhukhan, J. Chen, Z. Chen, V. Tiwari, R. Lai, J. Shi, I. E.-H. Yen, A. May, T. Chen, and B. Chen, "Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[279] T. Liu, Y. Li, Q. Lv, K. Liu, J. Zhu, W. Hu, and X. Sun, "PEARL: Parallel speculative decoding with adaptive draft length," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[280] L. Zhang, Z. Zhang, Xubaizhou, R. Li, Z. Tian, S. Mei, and D. Li, "Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 17 382–17 395.

[281] J. Cai, A. Vora, R. Zhang, M. O'Connor, and M. C. Jeffrey, "Attention-level speculation," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 6291–6318.

[282] Y. Wu, K. Gao, L. Li, and Y. Wu, "Easyspec: Layer-parallel speculative decoding for efficient multi-gpu utilization," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 158 879–158 903.

[283] K. Huang, H. Wu, Z. Shi, H. Zou, M. Yu, and Q. Shi, "Adaspec: Adaptive speculative decoding for fast, slo-aware large language model serving," in *Proceedings of the 2025 ACM Symposium on Cloud Computing*, 2025, pp. 361–374.

[284] P.-S. Wang, J.-J. Chen, C.-C. Yang, C.-C. Chang, N.-C. Huang, M. Abdelfattah, and K.-C. Wu, "Speculate deep and accurate: Lossless and training-free acceleration for offloaded llms via substitute speculative decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 54 334–54 357.

[285] N. Bhendawade, K. Nishu, A. Kundu, C. Bartels, M. Cho, and I. Belousova, "Mirror speculative decoding: Breaking the serial barrier in llm inference," *ArXiv preprint*, vol. arXiv:2510.13161, 2025.

[286] Y. Ma, L. Du, L. Wei, K. Chen, Q. Xu, K. Wang, G. Feng, G. Lu, L. Liu, X. Qi, X. Zhang, Z. Tao, H. Feng, Z. Jiang, Y. Xu, Z. Huang, Y. Zhuang, H. Xu, J. Hu, Z. Lan, J. Zhao, J. Li, and D. Zheng, "dinfer: An efficient inference framework for diffusion language models," *ArXiv preprint*, vol. arXiv:2510.08666, 2025.

[287] J. Fan, Y. Zhang, X. Li, and D. S. Nikolopoulos, "Taming the memory footprint crisis: System design for production diffusion llm serving," *ArXiv preprint*, vol. arXiv:2512.17077, 2025.

[288] J. Yao, K. Chen, K. Zhang, J. You, B. Yuan, Z. Wang, and T. Lin, "DeFT: Decoding with flash tree-attention for efficient tree-structured LLM inference," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[289] J. Zhao, W. Lu, S. Wang, L. Kong, and C. Wu, "QSpec: Speculative decoding with complementary quantization schemes," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 4779–4795.

[290] R. Tiwari, H. Xi, A. Tomar, C. R. C. Hooper, S. Kim, M. Horton, M. Najibi, M. W. Mahoney, K. Keutzer, and A. Gholami, "QuantSpec: Self-speculative decoding with hierarchical quantized KV cache," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 59 668–59 686.

[291] J. Liu, B. Chen, and C. Zhang, "Speculative prefill: Turbocharging TTFT with lightweight and training-free token importance estimation," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 38 188–38 209.

[292] E. Georganas, D. Kalamkar, A. Kozlov, and A. Heinecke, "Ml-specqd: Multi-level speculative decoding with quantized drafts," *ArXiv preprint*, vol. arXiv:2503.13565, 2025.

[293] S. Jie, Y. Tang, K. Han, Z.-H. Deng, and J. Han, "SpeCache: Speculative key-value caching for efficient generation of LLMs," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 27 917–27 928.

[294] Z. Huang, L. Zhu, Z. Zhan, T. Hu, W. Mao, X. Yu, Y. Liu, and T. Zhang, "Moesd: Unveil speculative decoding's potential for accelerating sparse moe," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 125 276–125 311.

[295] A. Saxena, P.-A. Tsai, H. Taneja, A. Jaleel, and M. Qureshi, "Utility-driven speculative decoding for mixture-of-experts," *ArXiv preprint*, vol. arXiv:2506.20675, 2025.

[296] S. Luo, Y. Guan, P. Li, H. Wang, and T. Chen, "Asyncspade: Efficient test-time scaling with asynchronous sparse decoding," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[297] Y. Guan, C. Yu, S. Fang, W. Hu, Z. Pan, Z. Wang, Z. Liu, Y. Zhou, Y. Ding, M. Guo, and J. Leng, "Yggdrasil: Bridging dynamic speculation and static runtime for latency-optimal tree-based llm decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 86 706–86 726.

[298] J. K. Christopher, B. R. Bartoldson, T. Ben-Nun, M. Cardei, B. Kailkhura, and F. Fioretto, "Speculative diffusion decoding: Accelerating language generation through diffusion," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 12 042–12 059.

[299] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang, "Diffuspec: Unlocking diffusion language models for speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 20 896–20 910.

[300] J. Sandler, J. Christopher, T. Hartvigsen, and F. Fioretto, "Specdiff-2: Scaling diffusion drafter alignment for faster speculative decoding," in *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026, pp. 1128–1147.

[301] Z. Cheng, G.-W. Yang, J. Li, Z. Deng, M.-H. Guo, and S.-M. Hu, "Deer: Draft with diffusion, verify with autoregressive models," *ArXiv preprint*, vol. arXiv:2512.15176, 2025.

[302] X. Gao, W. Xie, Y. Xiang, and F. Ji, "Falcon: Faster and parallel inference of large language models through enhanced semi-autoregressive drafting and custom-designed decoding tree," in *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI)*, 2025, pp. 23 933–23 941.

[303] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block diffusion for flash speculative decoding," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026, p. to appear.

[304] F. Liu, X. Li, K. Zhao, Y. Gao, Z. Zhou, Z. Zhang, Z. Wang, W. Dou, S. Zhong, and C. Tian, "Dart:

Diffusion-inspired speculative decoding for fast LLM inference," *ArXiv preprint*, vol. arXiv:2601.19278, 2026.

[305] X. Cheng, X. Yu, C. Shao, J. Li, Y. Xiong, Y. Qian, J. Zhu, S. Ma, X. Zhang, J. Ye, Q. Chen, C. Deng, J. Yu, D. Dai, Z. Zhang, Y. Wei, Y. Tan, W. Yang, R. Xu, Y. Wu, Z. Xu, X. Wang, M. Chen, R. Tian, X. Bi, Z. Hao, S. Chen, H. Cao, W. Zhang, A. Xu, H. Zhang, D. Zhao, and W. Liang, "Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation," *ArXiv preprint*, vol. arXiv:2607.05147, 2026.

[306] Y.-Y. Qian, H.-C. Wu, C. Chen, J. Sun, Z. Dong, P. Zhao, and Z.-H. Zhou, "Adaflash: Adaptive speculative decoding via on-policy distilled diffusion drafters," *ArXiv preprint*, vol. arXiv:2607.19223, 2026.

[307] D. Israel, G. Van den Broeck, and A. Grover, "Accelerating diffusion llms via adaptive parallel decoding," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 52 870–52 888.

[308] Y. Gao, Z. Ji, Y. Wang, B. Qi, H. Xu, and L. Zhang, "Self speculative decoding for diffusion large language models," *ArXiv preprint*, vol. arXiv:2510.04147, 2025.

[309] S. Agrawal, R. Garrepalli, R. Goel, C. Lott, F. Porikli, and M. Lee, "Structuring the future: Diffusion LLM speculative decoding via calibrated draft graphs," in *The 1st FoGen Workshop, ICML 2026*, 2026.

[310] L. Wei, W. Chen, P. Tang, X. Guo, L. Ye, R. Wang, and M. Li, "Orchestrating dual-boundaries: An arithmetic intensity inspired acceleration framework for diffusion language models," in *Proceedings of the 63rd ACM/IEEE Design Automation Conference (DAC)*, 2026.

[311] Z. Chen, X. Yang, J. Lin, C. Sun, K. C.-C. Chang, and J. Huang, "Cascade speculative drafting for even faster llm inference," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 86 226–86 242.

[312] H. Narasimhan, W. Jitkrittum, A. S. Rawat, S. Kim, N. Gupta, A. K. Menon, and S. Kumar, "Faster cascades via speculative decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[313] Y. Teng, H. Shi, X. Liu, X. Ning, G. Dai, Y. Wang, Z. Li, and X. Liu, "Accelerating auto-regressive text-to-image generation with training-free speculative jacobi decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[314] D. Jang, S. Park, J. Y. Yang, Y. Jung, J. Yun, S. Kundu, S.-Y. Kim, and E. Yang, "LANTERN: Accelerating visual autoregressive models with relaxed speculative decoding," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[315] V. De Bortoli, A. Galashov, A. Gretton, and A. Doucet, "Accelerated diffusion models via speculative sampling," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 12 590–12 631.

[316] H. Hu, A. Das, D. Sadigh, and N. Anari, "Diffusion models are secretly exchangeable: Parallelizing DDPMs via auto speculation," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 24 270–24 289.

[317] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 7655–7671.

[318] L. Zhang, L. Fang, C. Duan, M. He, L. Pan, P. Xiao, S. Huang, Y. Zhai, X. Hu, P. S. Yu, and A. Liu, "A survey on parallel text generation: From parallel decoding to diffusion language models," *ArXiv preprint*, vol. arXiv:2508.08712, 2025.

[319] Z. Wang, B. Niu, Z. Gao, Z. Zheng, T. Xu, L. Meng, Z. Li, J. Liu, Y. Chen, C. Zhu, H. Wu, H. Wang, and E. Chen, "A survey on parallel reasoning," *ArXiv preprint*, vol. arXiv:2510.12164, 2025.

[320] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Spec-bench: A comprehensive benchmark and unified evaluation platform for speculative decoding," https://sites.google.com/view/spec-bench/, 2024.

[321] S. Sun, Y. Li, X. Li, Y. Lian, W. Lin, H.-L. Zhen, Z. Yang, X. Yu, C. Chen, M. Yuan, and C. Ma, "Scaling up, speeding up: A benchmark of speculative decoding for efficient LLM test-time scaling," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[322] H. Peng, P. Liu, Z. Dong, D. Cheng, J. Li, Y. Tang, S. Wang, and W. X. Zhao, "How efficient are diffusion language models? a critical examination of efficiency evaluation practices," *ArXiv preprint*, vol. arXiv:2510.18480, 2025.

[323] W. Kang, K. Galim, S. Oh, M. Lee, Y. Zeng, S. Zhang, C. Hooper, Y. Hu, H. I. Koo, N. I. Cho, and K. Lee, "Parallelbench: Understanding the trade-offs of parallel decoding in diffusion LLMs," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026.

[324] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 11 809–11 822.

[325] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[326] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang *et al.*, "Qwen2.5 technical report," *ArXiv preprint*, vol. arXiv:2412.15115, 2024.

[327] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, "Training verifiers to solve math word problems,"

ArXiv preprint, vol. arXiv:2110.14168, 2021.

[328] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[329] Y. Zhang and Math-AI Team, "American invitational mathematics examination (AIME) 2025," https://huggingface.co/datasets/math-ai/aime25, 2025.

[330] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *ArXiv preprint*, vol. arXiv:2107.03374, 2021.

[331] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *ArXiv preprint*, vol. arXiv:2108.07732, 2021.

[332] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 46 595–46 623.

[333] T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica, "From crowdsourced data to high-quality benchmarks: Arena-Hard and BenchBuilder pipeline," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 34 209–34 231.